



中国科学技术大学
University of Science and Technology of China

计算系统概论A

Introduction to Computing Systems
(CS1002A.03)

Chapter 5 The LC-3

陈俊仕

cjuns@ustc.edu.cn

2025 Fall

计算机科学与技术学院

School of Computer Science and Technology



运算指令

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD	0	0	0	1	DR			SR1			0	0	0	SR2		
ADD	0	0	0	1	DR			SR1			1	Imm5				
AND	0	1	0	1	DR			SR1			0	0	0	SR2		
AND	0	1	0	1	DR			SR1			1	Imm5				
NOT	1	0	0	1	DR			SR1			1	1	1	1	1	1
Reserved	1	1	0	1												

控制指令

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR	0	0	0	0	n	z	p	PCoffset9								
JSR	0	1	0	0	1	PCoffset11										
JSRR	0	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	0
RTI	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
JMP	1	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	0
RET	1	1	0	0	0	0	0	1	1	1	0	0	0	0	0	0
TRAP	1	1	1	1	0	0	0	0	TrapVector8							

移动数据指令
取数指令

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LD	0	0	1	0	DR			PCOffset9								
LDR	0	1	1	0	DR			BaseR		PCOffset6						
LDI	1	0	1	0	DR			PCOffset9								
LEA	1	1	1	0	DR			PCOffset9								

存数指令

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ST	0	0	1	1	SR			PCoffset9								
STR	0	1	1	1	SR			BaseR			PCoffset6					
STI	1	0	1	1	SR			PCoffset9								

1 LC-3 ISA Overview

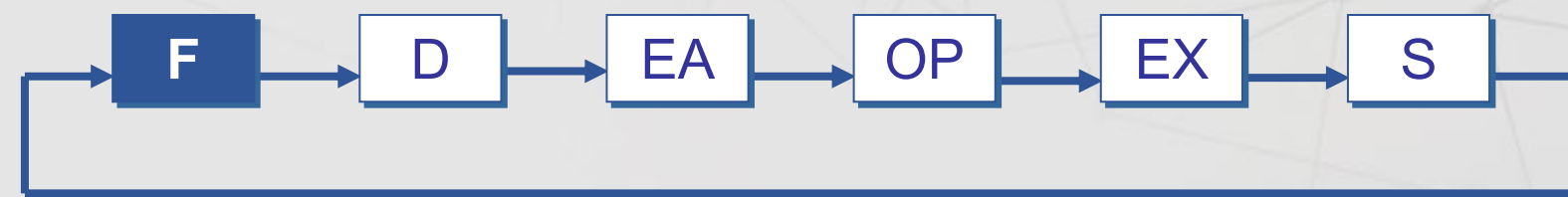
2 Operate Instructions and Data Path

3 Data Movement Instructions and Data Path

4 Control Instructions and Data Path

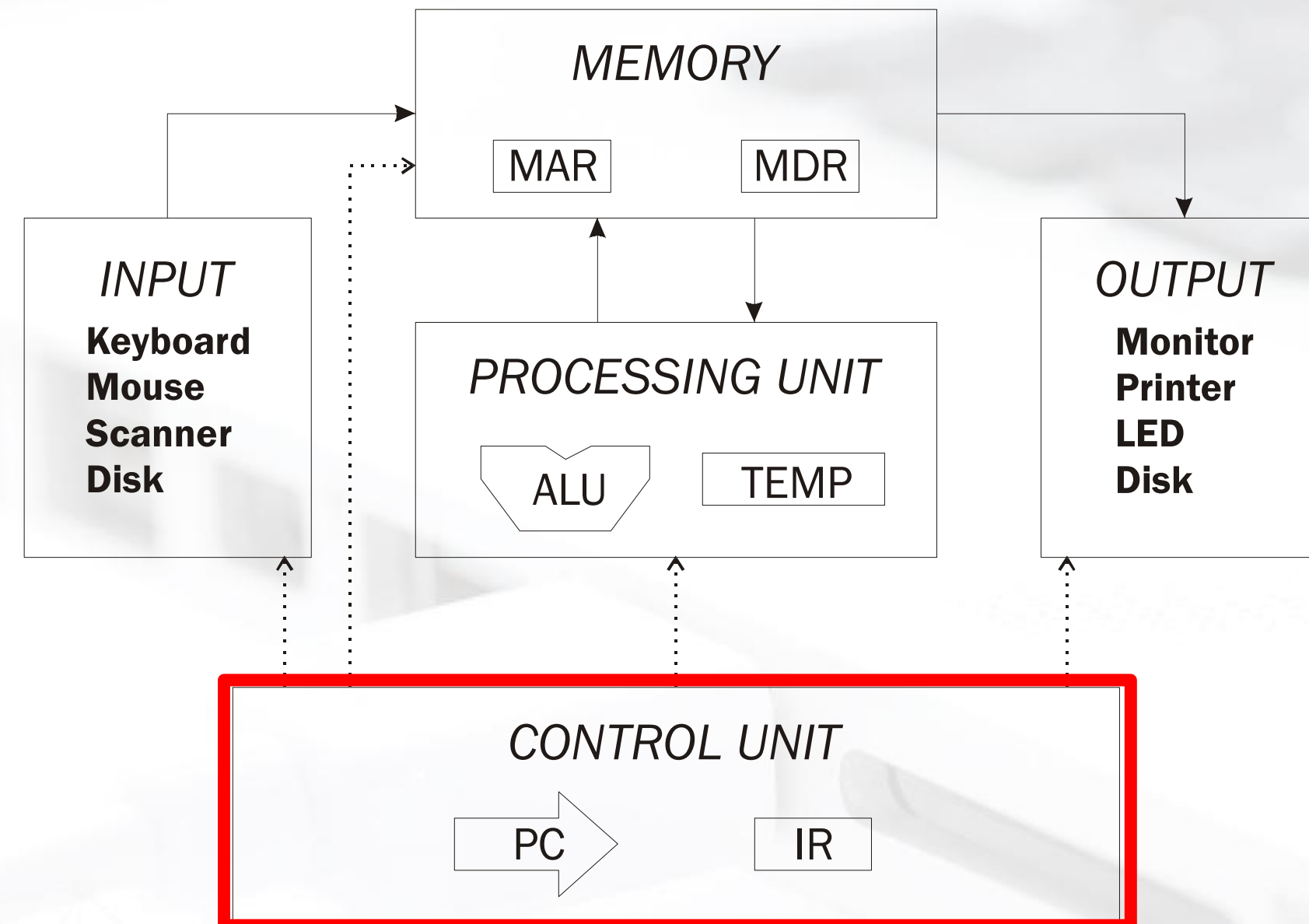
5 Another Example: Counting Occurrences of a Computer

6 The Data Path Revisited



■ We are going to learn how to:

- Used to alter the sequence of instructions (by changing the Program Counter)



Control Instructions

Conditional Branch

- **branch is *taken* if a specified condition is true**
 - signed offset is added to PC to yield new PC
- **else, the branch is *not taken***
 - PC is not changed, points to the next sequential instruction

Unconditional Branch (or Jump)

- **always changes the PC**

TRAP

- **changes PC to the address of an OS “service routine”**
- **routine will return control to the next instruction**
(after TRAP)

Control Instructions



	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR	0	0	0	0	n	z	p	PCOffset9								
JSR	0	1	0	0	1	PCOffset11										
JSRR	0	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	0
RTI	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
JMP	1	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	0
RET	1	1	0	0	0	0	0	1	1	1	0	0	0	0	0	0
TRAP	1	1	1	1	0	0	0	0	TrapVector8							

Condition Codes

LC-3 has three **condition code** registers:

N -- negative

Z -- zero

P -- positive (greater than zero)

Set by any instruction that writes a value to a register
(ADD, AND, NOT, LD, LDR, LDI, LEA)

Exactly one will be set at all times

- Based on the last instruction that altered a register

Conditional Branch Instruction

Branch specifies one or more condition codes.

■ If the specified bit is set, the branch is taken.

- PC-relative addressing:

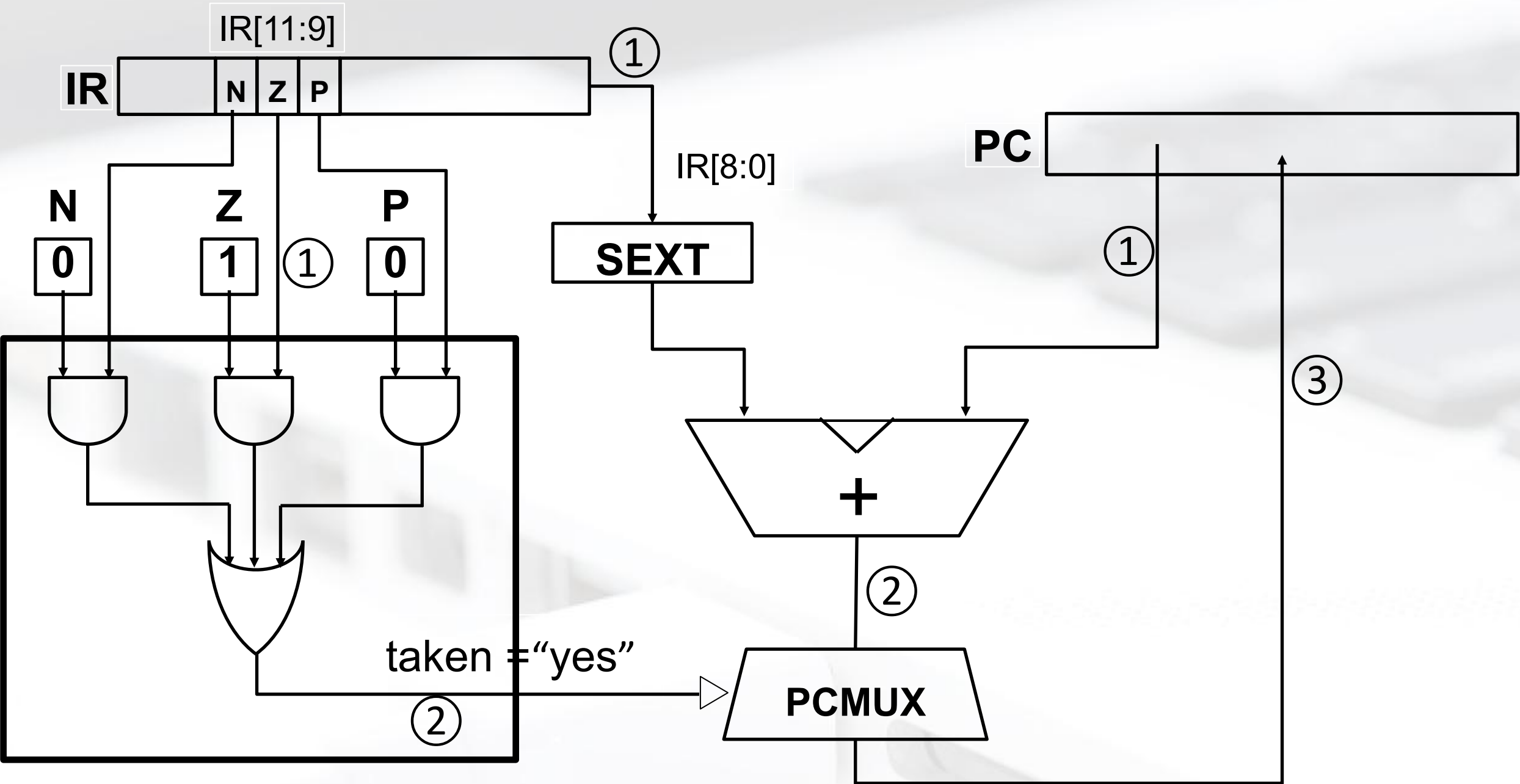
target address is made by adding signed offset ($IR[8:0]$) to current PC.

- Note: PC has already been incremented by FETCH stage.

- Note: Target must be within 256 words of BR instruction.

■ If the branch is not taken, the next sequential instruction is executed.

BR (PC-Relative)



What happens if bits [11:9] are all zero?
 What happens if bits [11:9] are all one?

BR (PC-Relative)

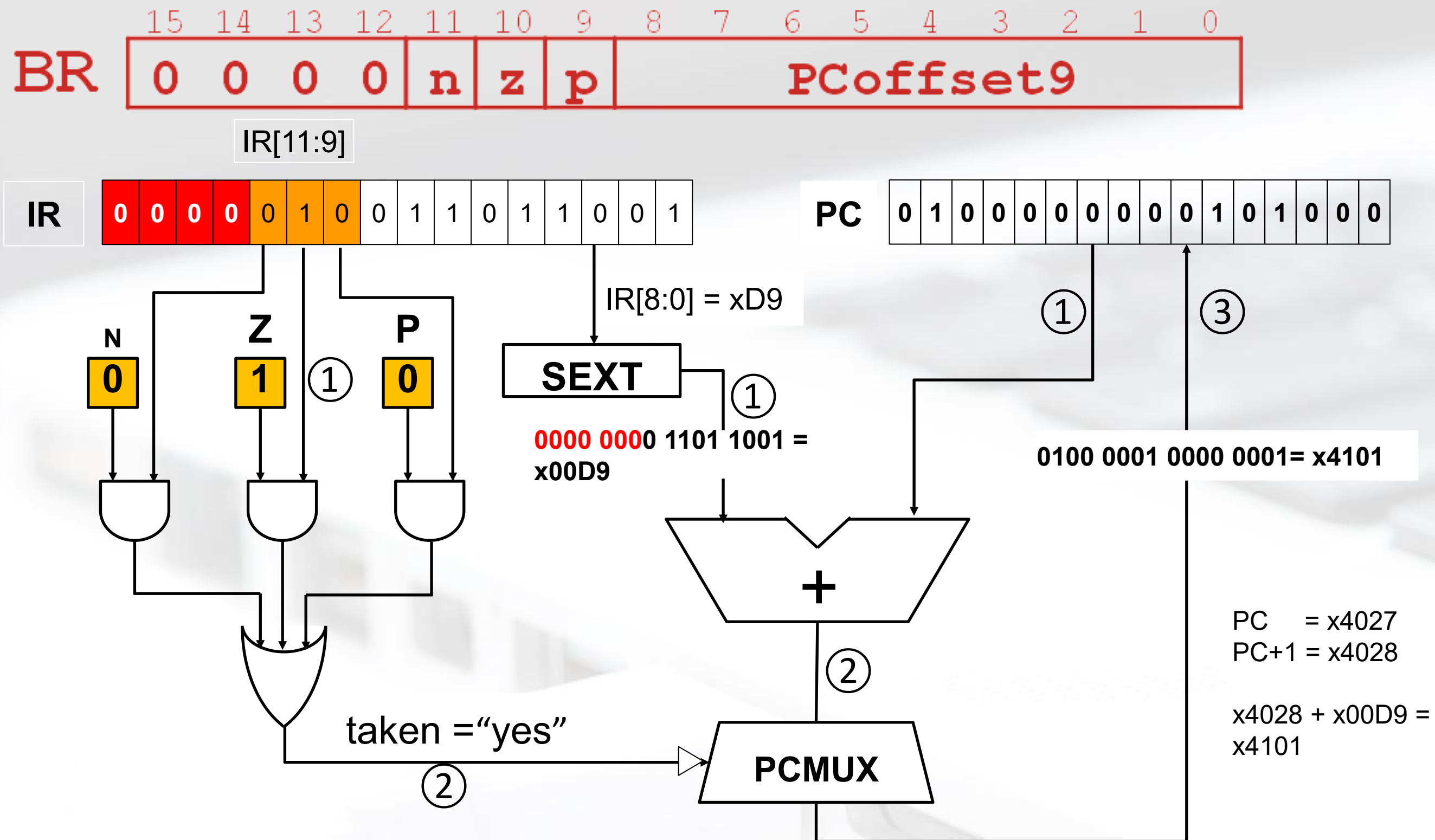
■ Check

- BR_{nzp} x4101 ; if ($n=1$ or $z=1$ or $p=1$) , JMP x4101
- BR_n x4101 ; if ($n=1$)
- BR_z x4101 ; if ($z=1$)
- BR_p x4101 ; if ($p=1$)
- BR_{nz} x4101 ; if ($n=1$ or $z=1$)
- BR_{np} x4101 ; if ($n=1$ or $p=1$)
- BR_{zp} x4101 ; if ($z=1$ or $p=1$)
- BR x4101 ; $PC=PC+1$

■ Set

- If $DR < 0$, set $N=1$ and $Z=0$ and $P=0$
- If $DR = 0$, set $N=0$ and $Z=1$ and $P=0$
- If $DR > 0$, set $N=0$ and $Z=0$ and $P=1$

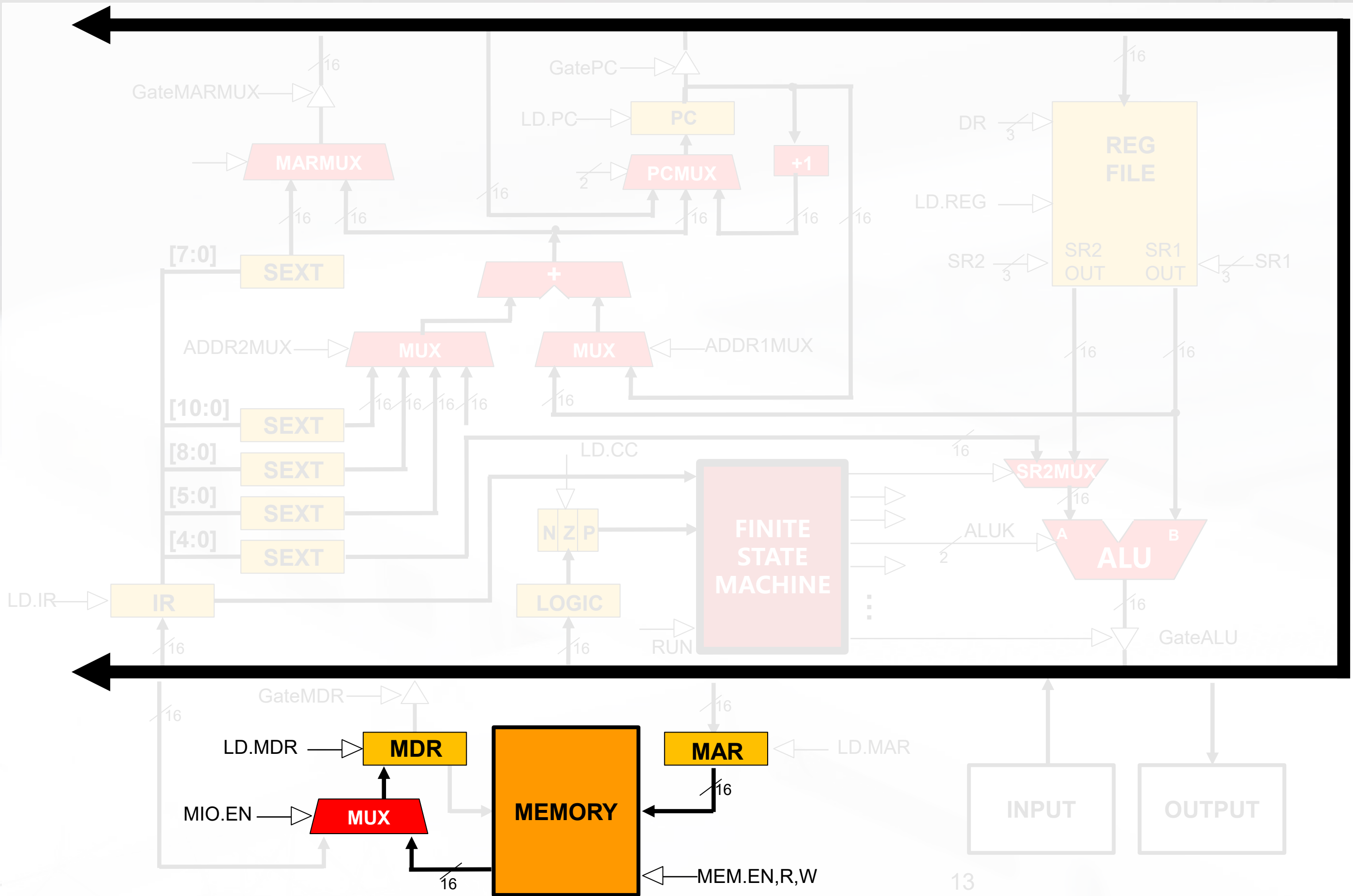
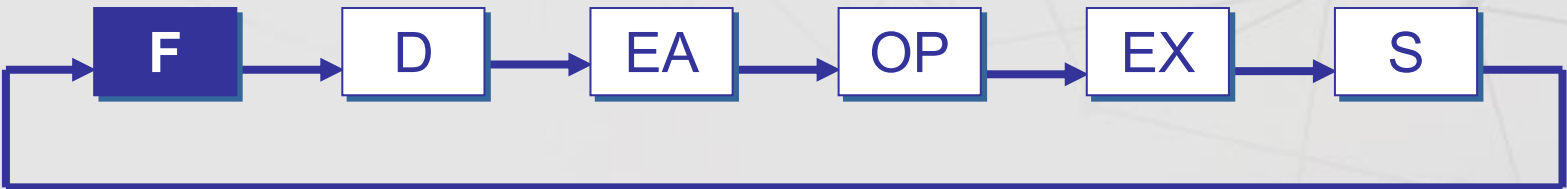
BR (PC-Relative): BR_z x4101



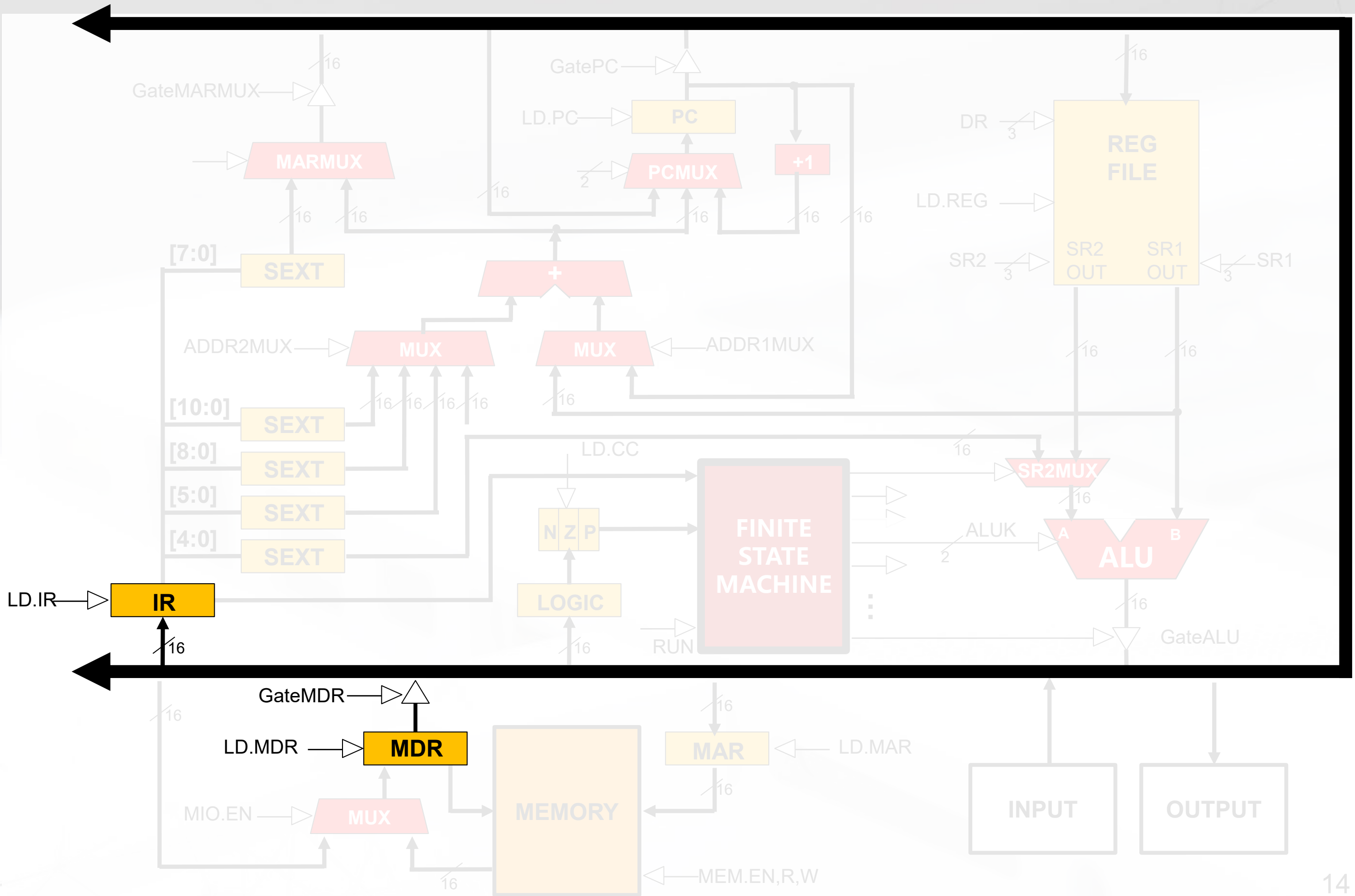
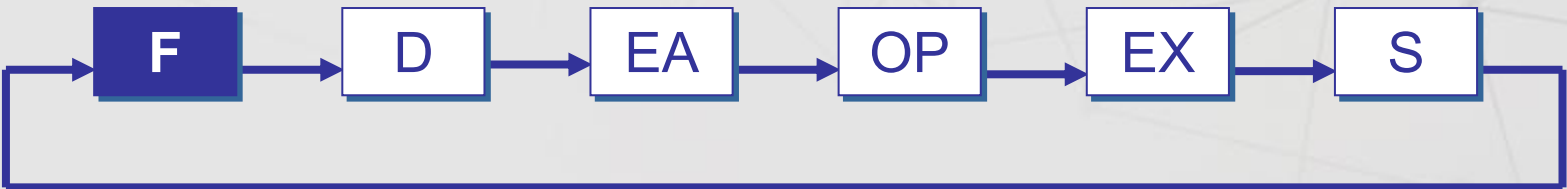
What happens if bits [11:9] are all zero?
What happens if bits [11:9] are all one?



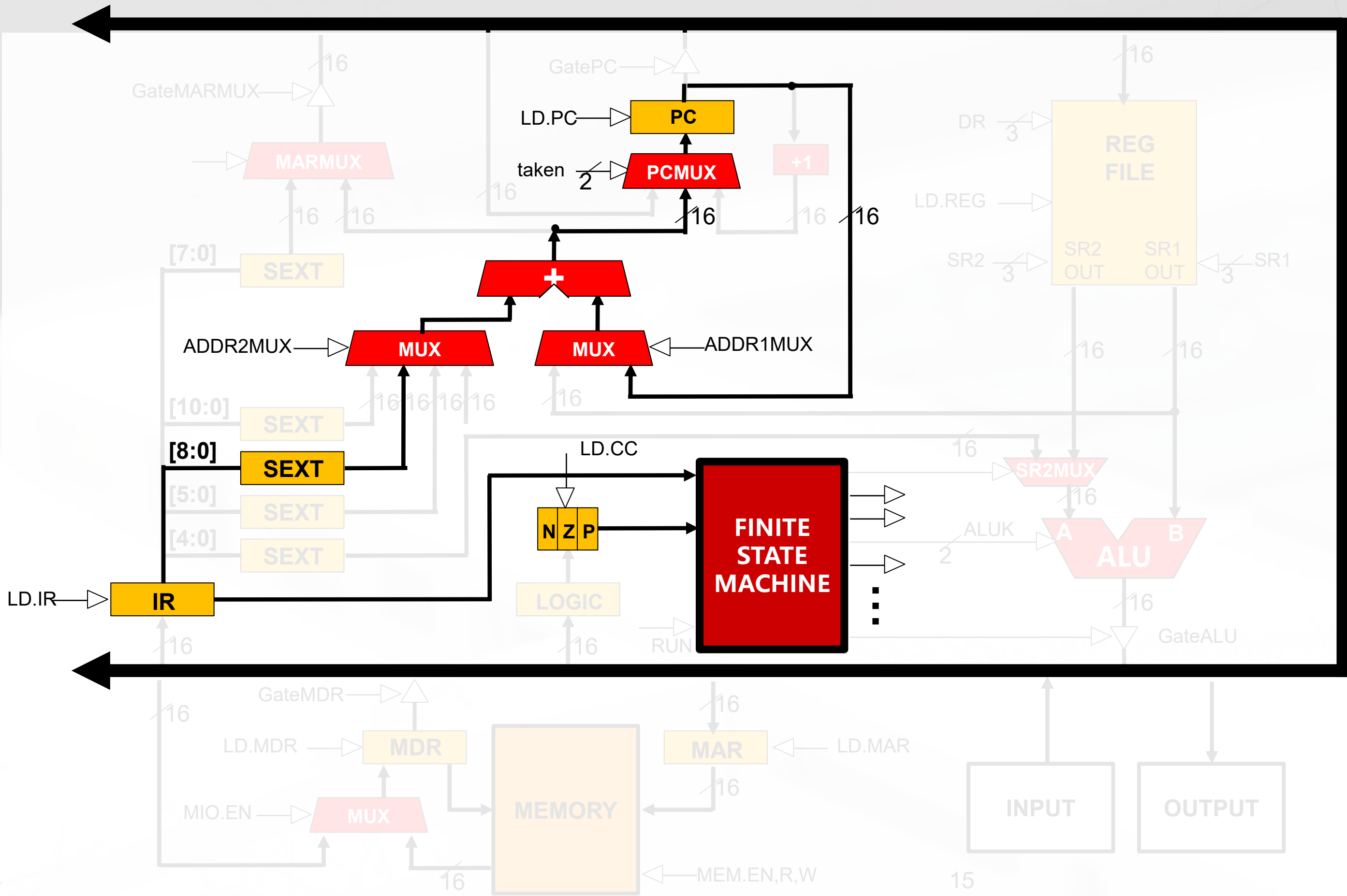
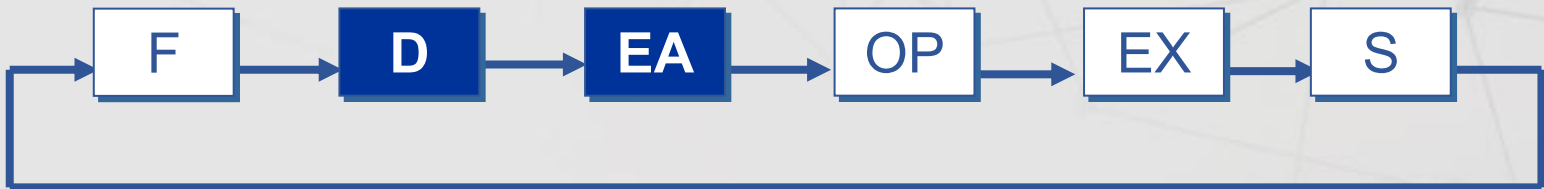
BR (PC-Relative)



BR (PC-Relative)



BR (PC-Relative)

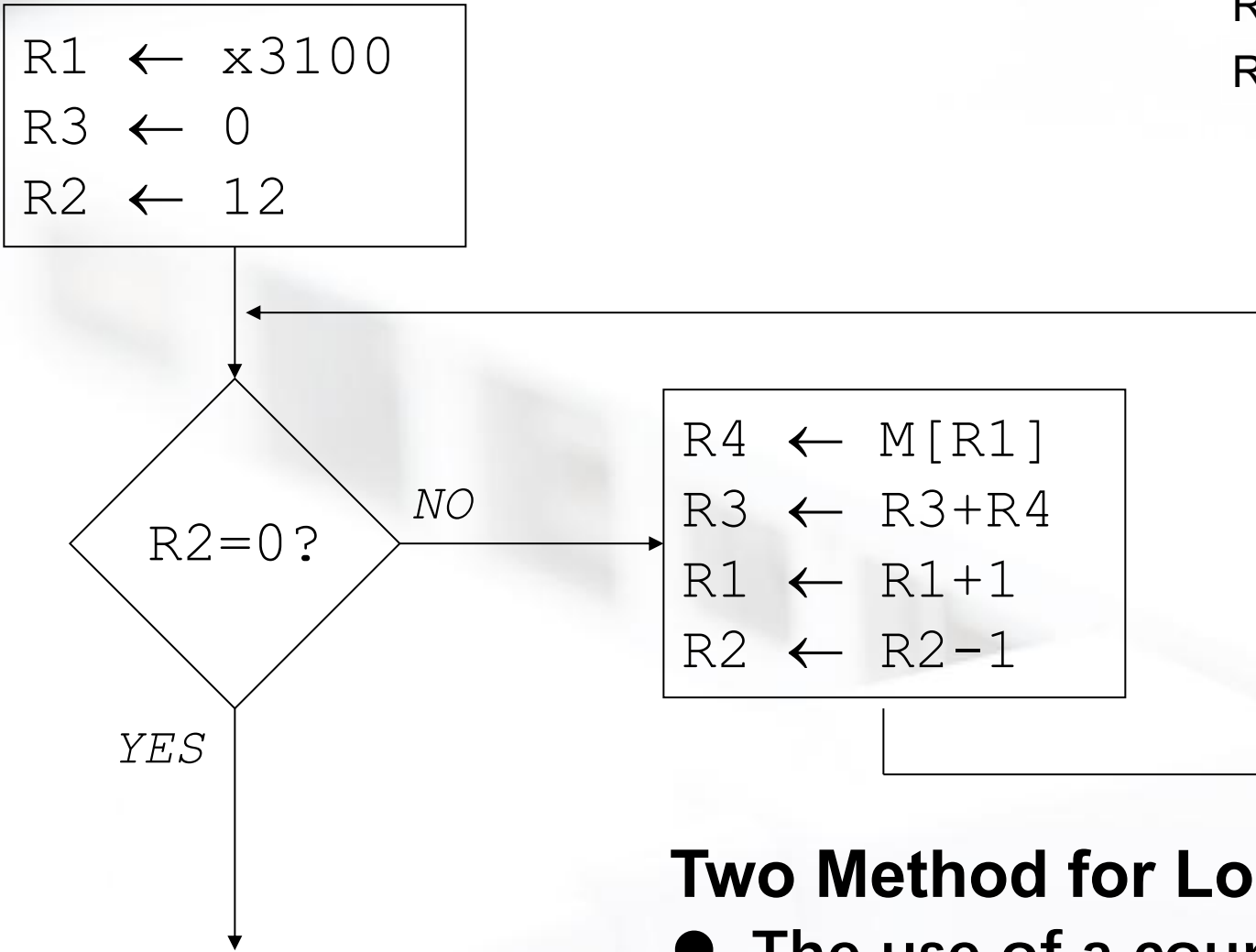


Using Branch Instructions



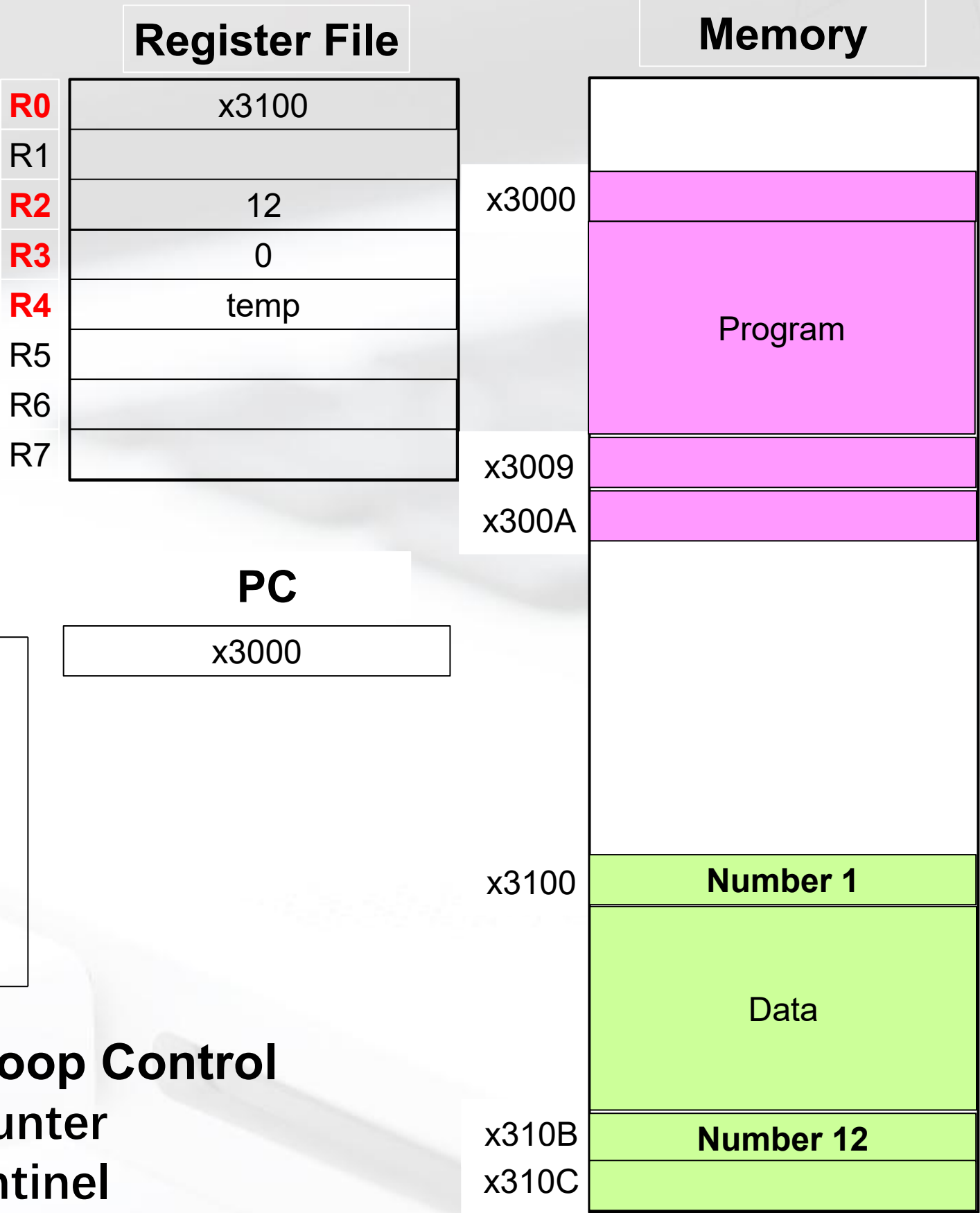
Compute sum of 12 integers.
Numbers start at location x3100.
Program starts at location x3000.

The use of a counter



Two Method for Loop Control

- The use of a counter
- The use of a sentinel



Sample Program(The use of a counter)

Address	Instruction															Comments
x3000	1	1	1	0	0	0	1	0	1	1	1	1	1	1	1	$R1 \leftarrow x3100 (PC+0xFF)$
x3001	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	$R3 \leftarrow 0$
x3002	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	$R2 \leftarrow 0$
x3003	0	0	0	1	0	1	0	0	1	0	1	0	1	1	0	$R2 \leftarrow 12$
x3004	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	If Z, goto (PC+5) = x300A
x3005	0	1	1	0	1	0	0	0	0	1	0	0	0	0	0	Load next value to R4
x3006	0	0	0	1	0	1	1	0	1	1	0	0	0	1	0	$R3 \leftarrow R3 + R4$
x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	Increment R1 (pointer)
x3008	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	Decrement R2 (counter)
x3009	0	0	0	0	1	1	1	1	1	1	1	1	1	0	1	Goto (PC-6)=x3004

Using Branch Instructions

Compute sum of 12 integers.

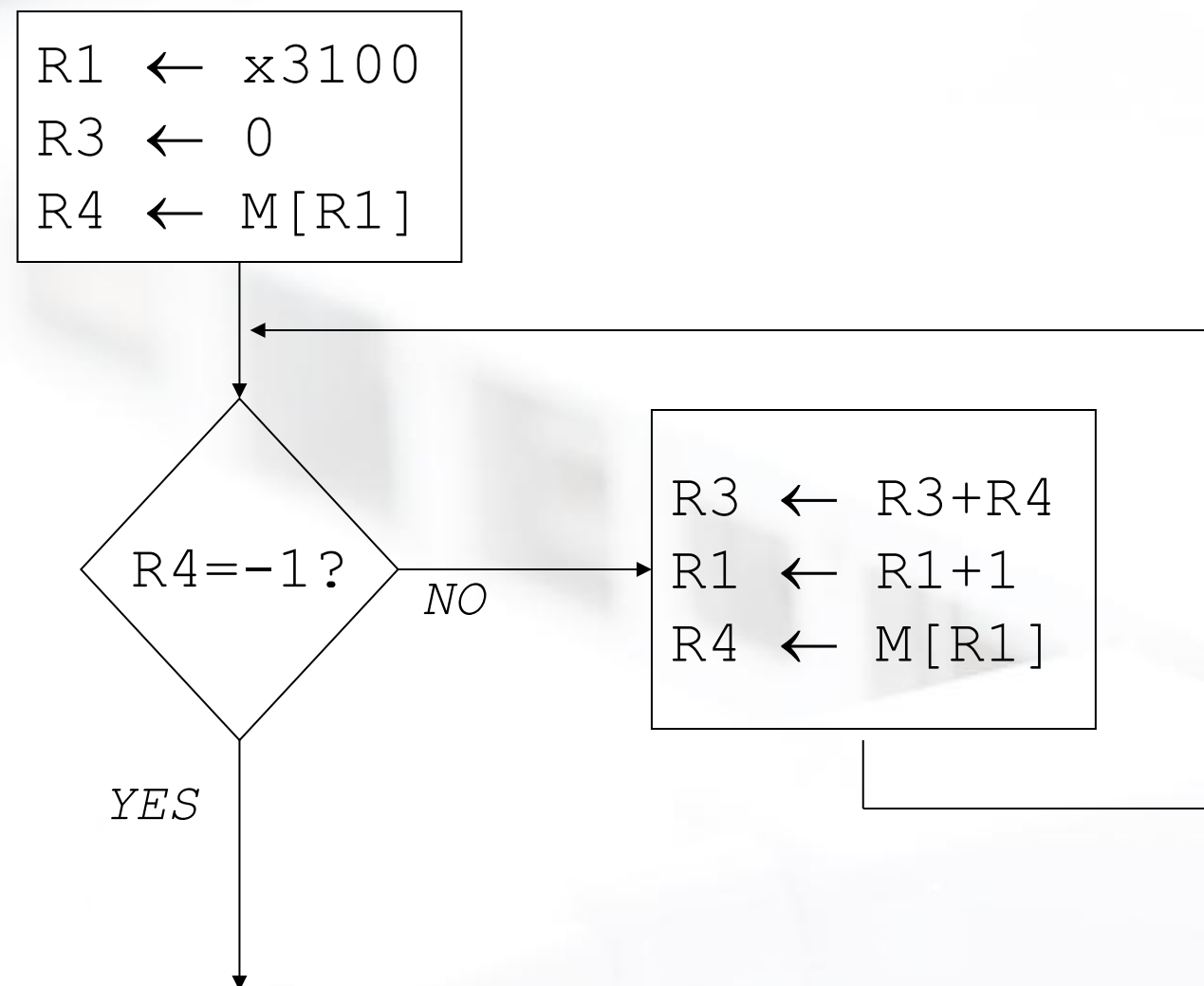
Numbers start at location x3100.

Program starts at location x3000.

A special character used to indicate the end of a sequence is often called a **sentinel**.

- Useful when you don't know ahead of time how many times to execute a loop.

The use of a sentinel

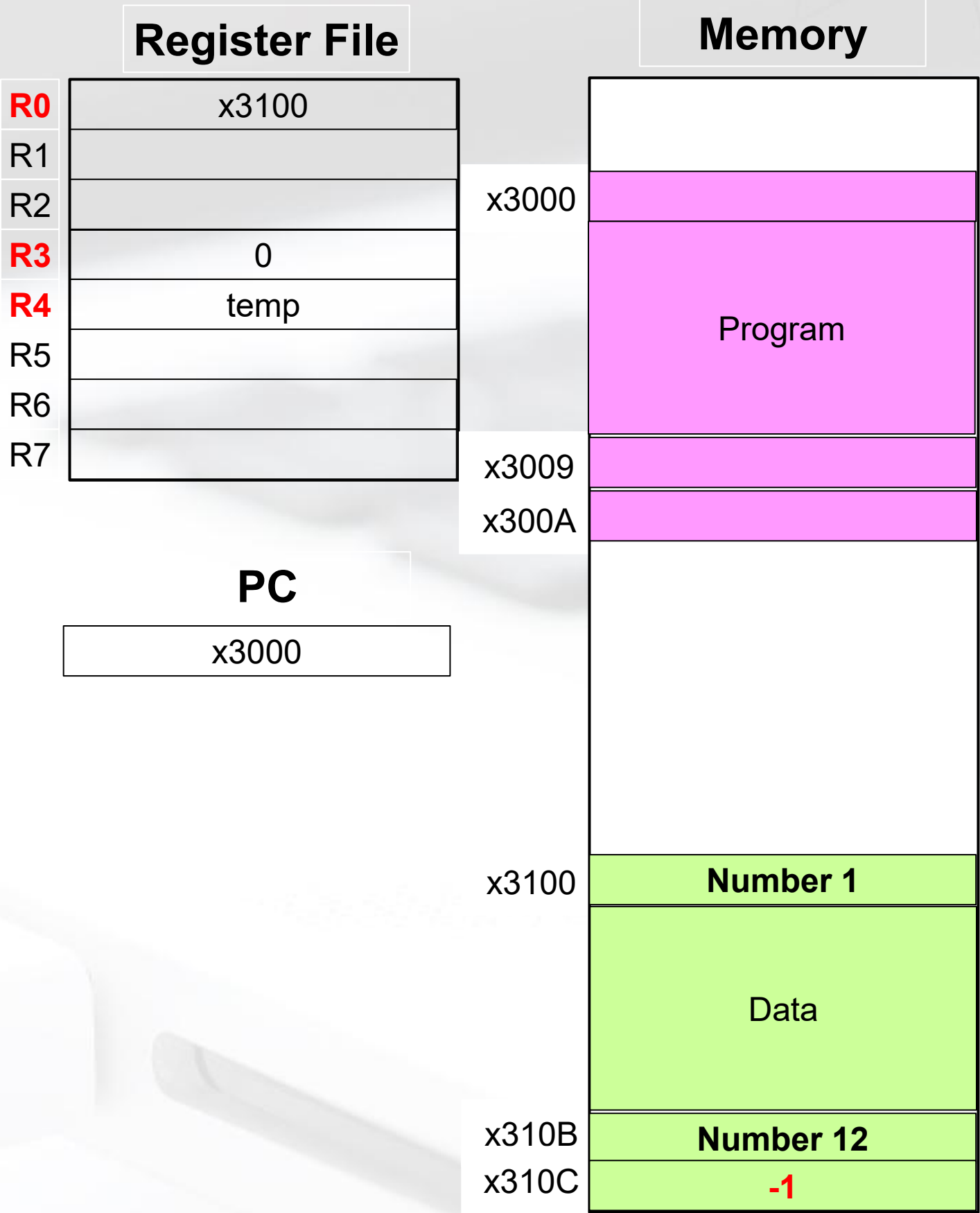
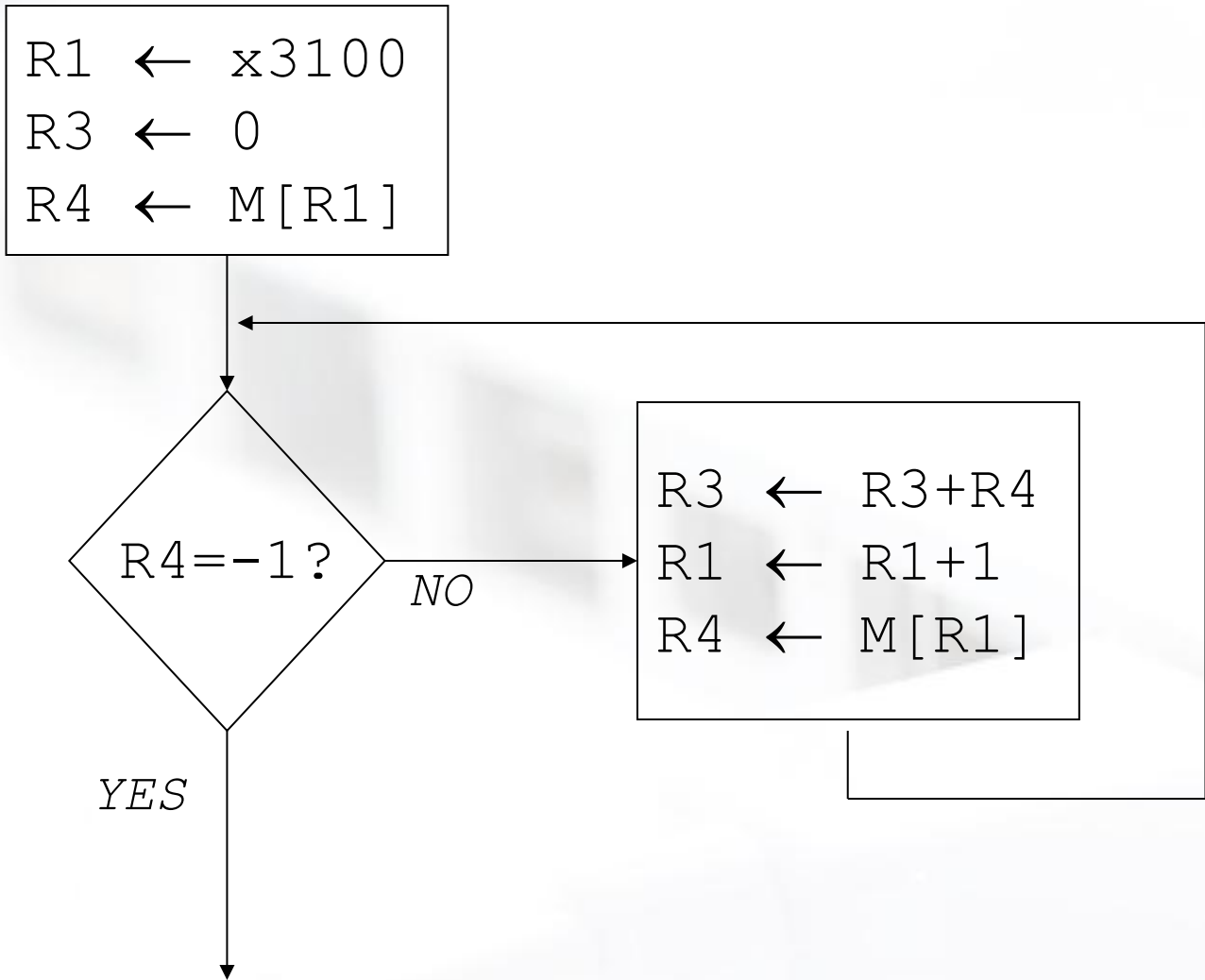


Using Branch Instructions



Compute sum of 12 integers.
Numbers start at location x3100.
Program starts at location x3000.

The use of a sentinel



Sample Program(The use of a sentinel)

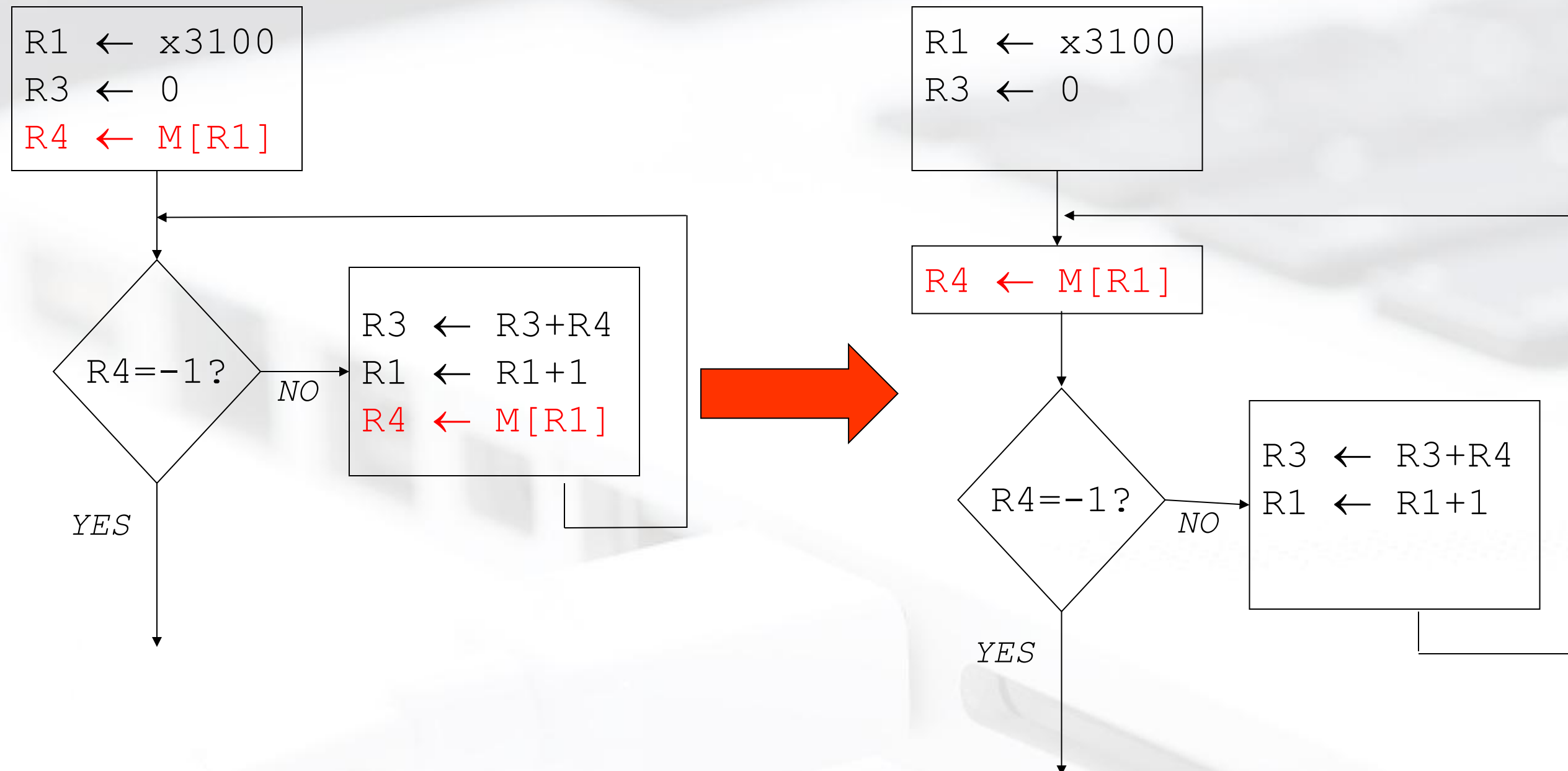
Address	Instruction														Comments
x3000	1	1	1	0	<u>0</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	$R1 \leftarrow (PC+0xFF)=x3100$
x3001	0	1	0	1	<u>0</u>	<u>1</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>1</u>	1	<u>0</u>	<u>0</u>	<u>0</u>	$R3 \leftarrow 0$
x3002	0	1	1	0	<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	$R4 \leftarrow M[R1]$
x3003	0	0	0	0	<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>0</u>	If N, goto (PC+ 4)=X3008
x3004	0	0	0	1	<u>0</u>	<u>1</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>1</u>	0	<u>0</u>	<u>0</u>	<u>1</u>	$R3 \leftarrow R3 + R4$
x3005	0	0	0	1	<u>0</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	$R1 \leftarrow R1 + 1$
x3006	0	1	1	0	<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	$R4 \leftarrow M[R1]$
x3007	0	0	0	0	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>0</u>	Goto (PC-5)= x3003

Using Branch Instructions(Code Optimization)

Compute sum of 12 integers.

Numbers start at location x3100.

Program starts at location x3000.



Sample Program



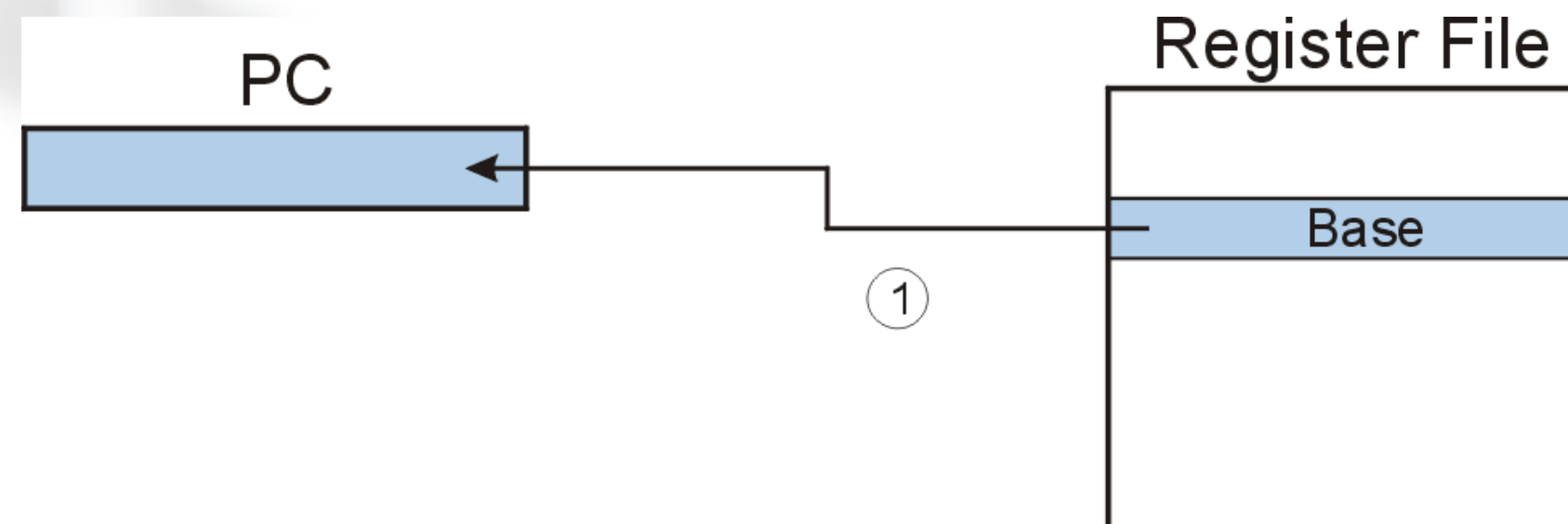
Address	Instruction															Comments
x3000	1	1	1	0	0	0	1	0	1	1	1	1	1	1	1	$R1 \leftarrow (PC+0xFF)= x3100$
x3001	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	$R3 \leftarrow 0$
x3002	0	1	1	0	1	0	0	0	0	1	0	0	0	0	0	$R4 \leftarrow M[R1]$
x3003	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	$\text{If } N, \text{ goto } (PC+ 3)=X3007$
x3004	0	0	0	1	0	1	1	0	1	1	0	0	0	1	0	$R3 \leftarrow R3 + R4$
x3005	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	$R1 \leftarrow R1 + 1$
x3006	0	0	0	0	1	1	1	1	1	1	1	1	1	0	1	$\text{Goto } (PC-5)= x3002$

JMP (Register)

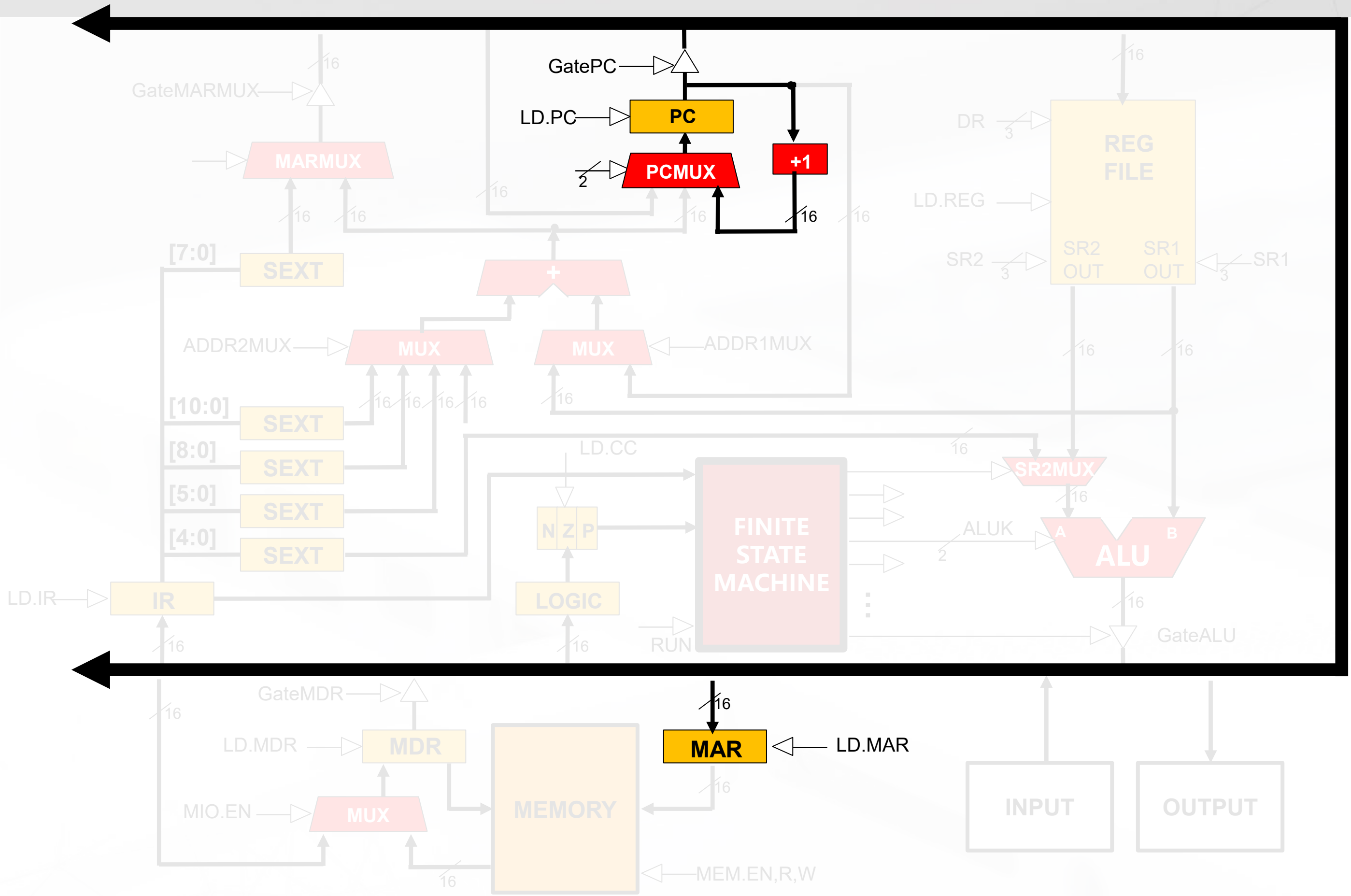
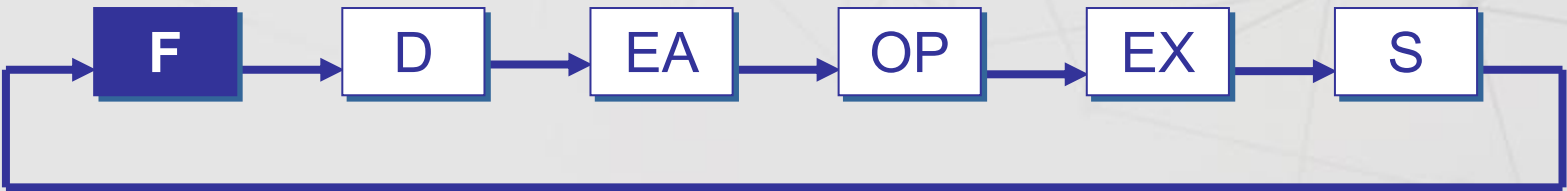
Jump is an unconditional branch -- always taken.

- Target address is the contents of a register.
- Allows any target address.

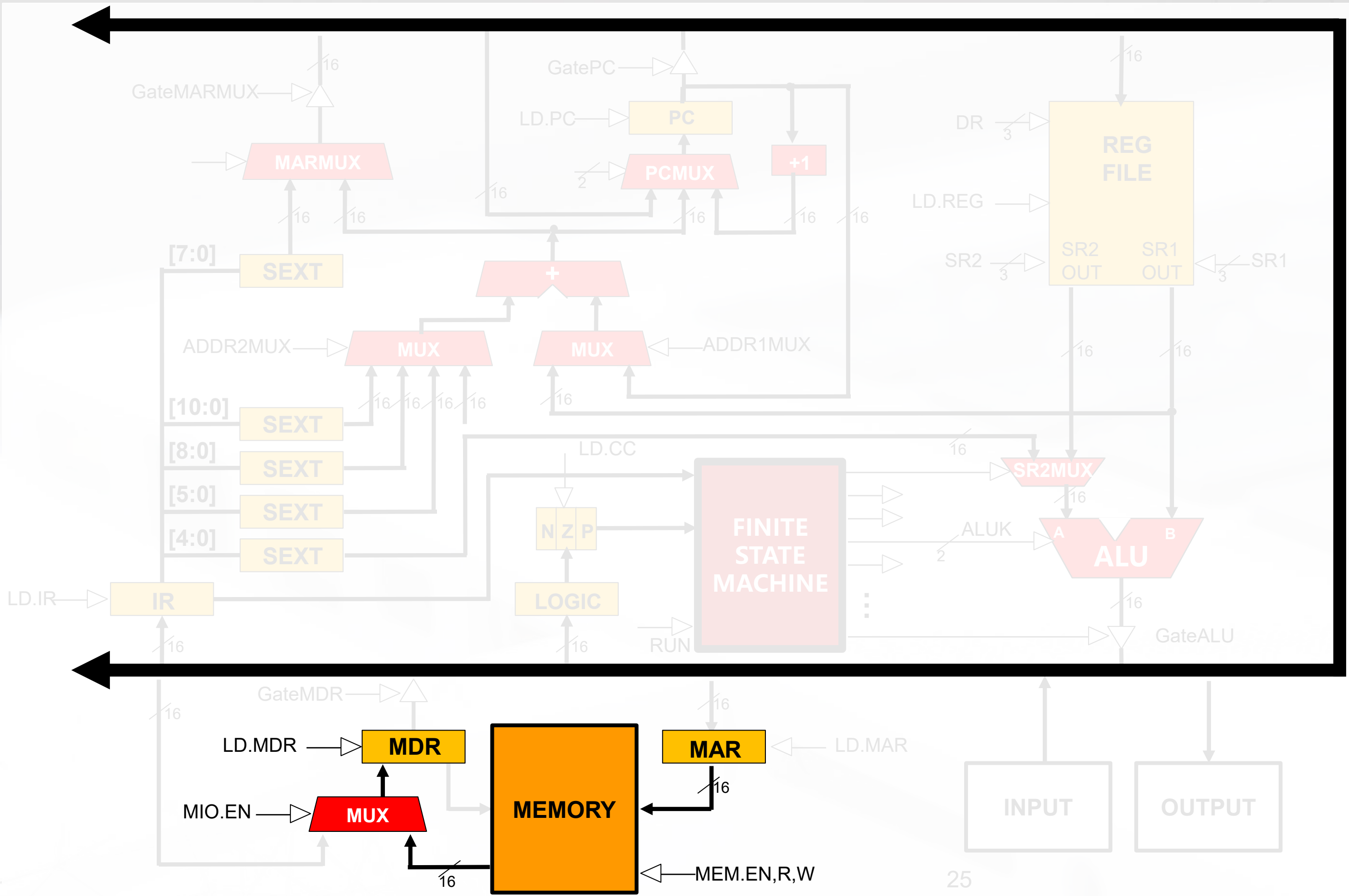
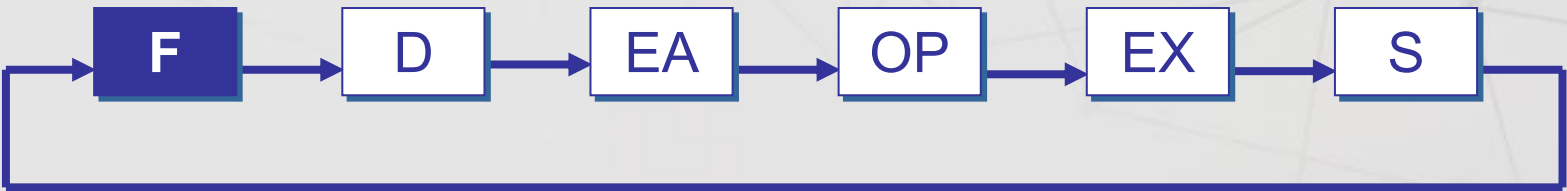
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JMP	1	1	0	0	0	0	0	Base			0	0	0	0	0	0



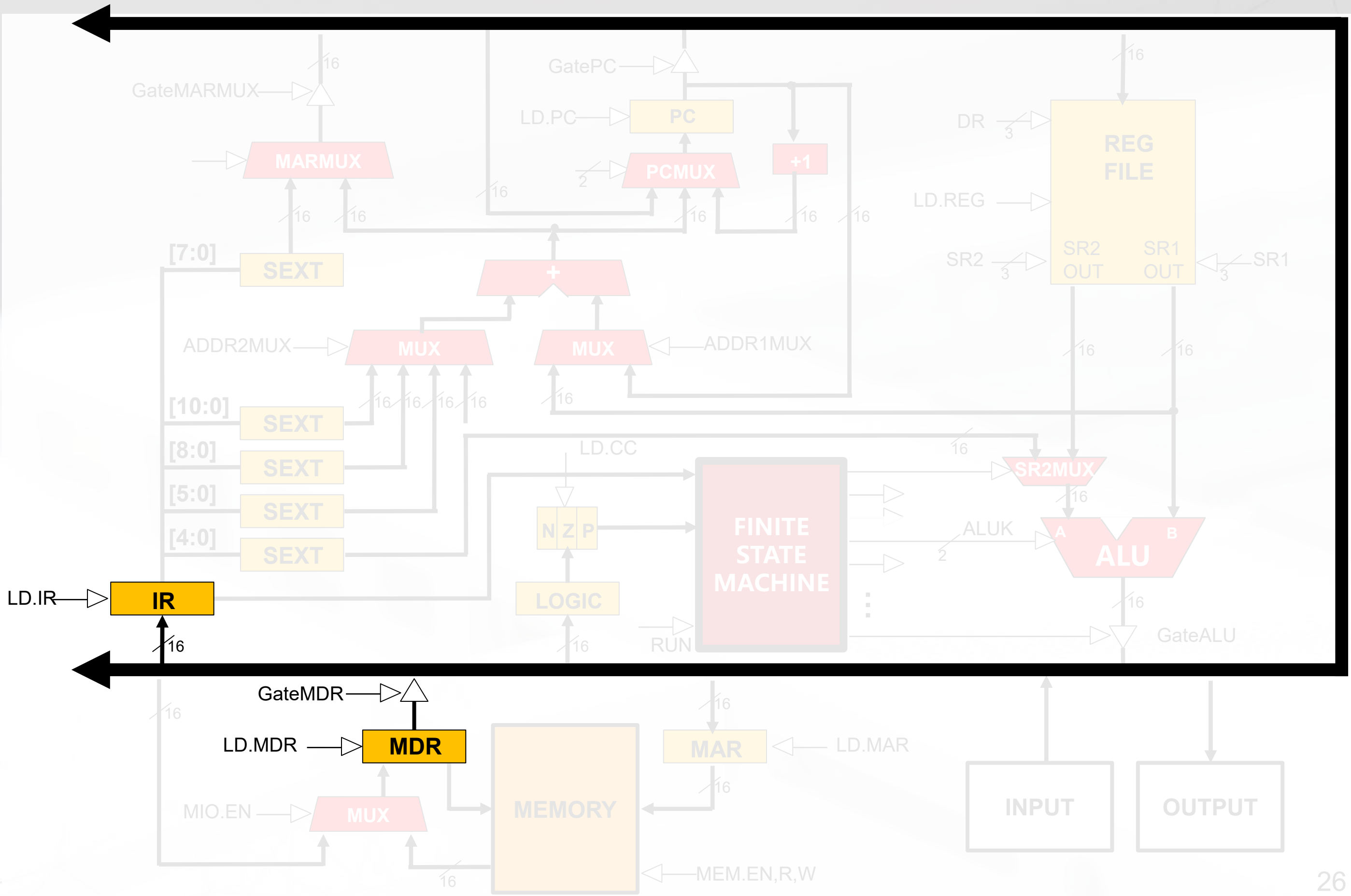
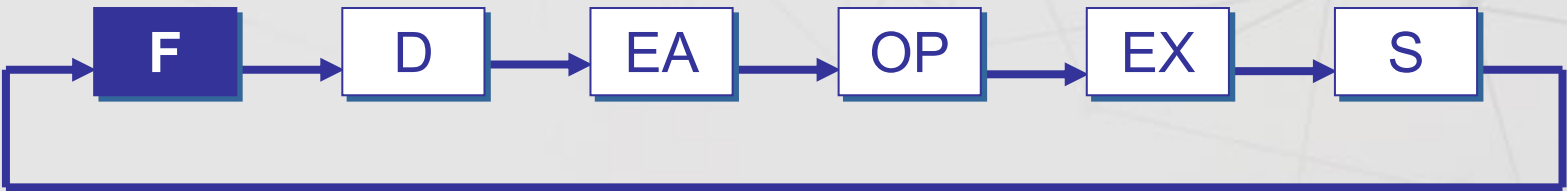
JMP (Register)



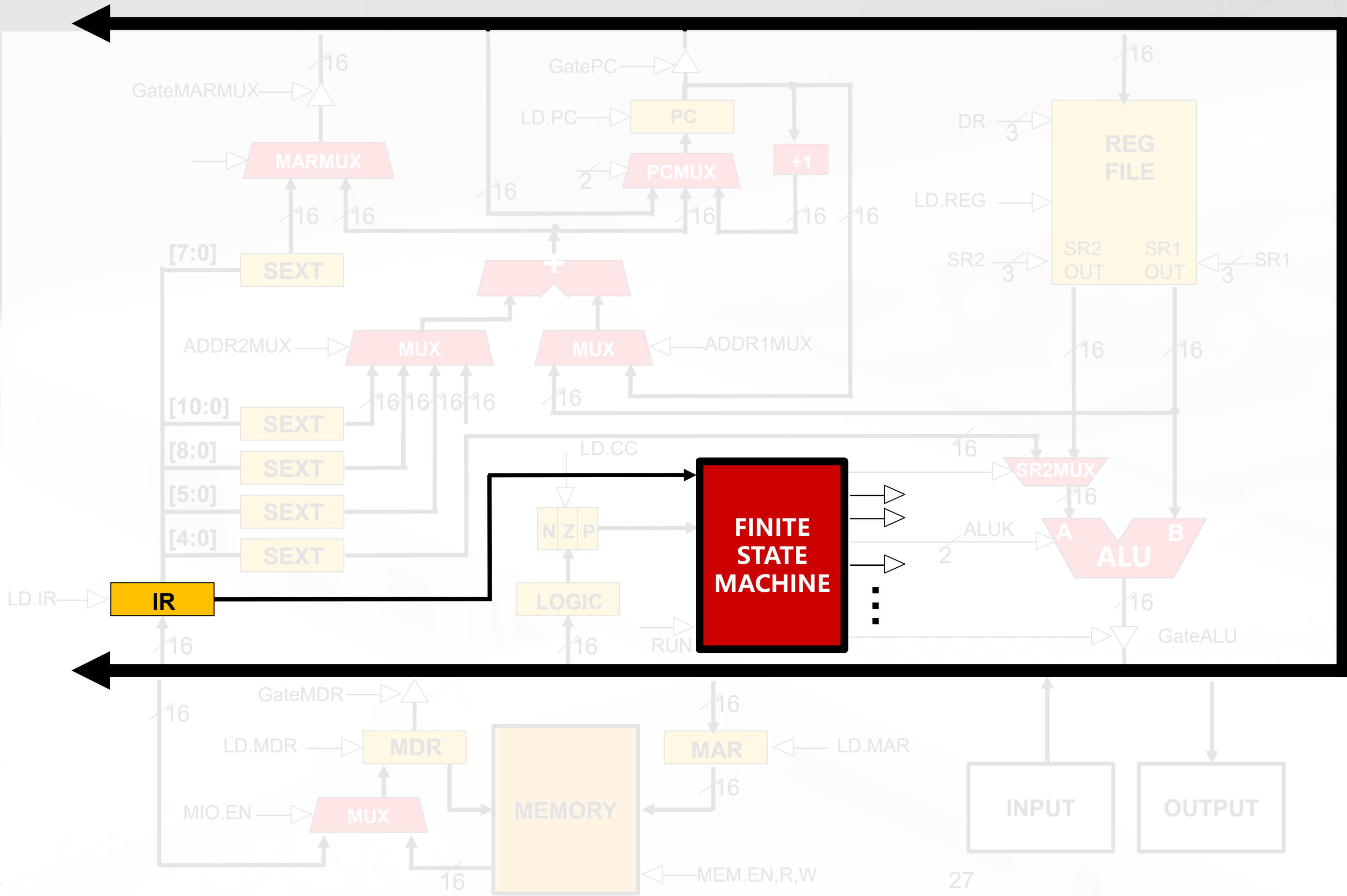
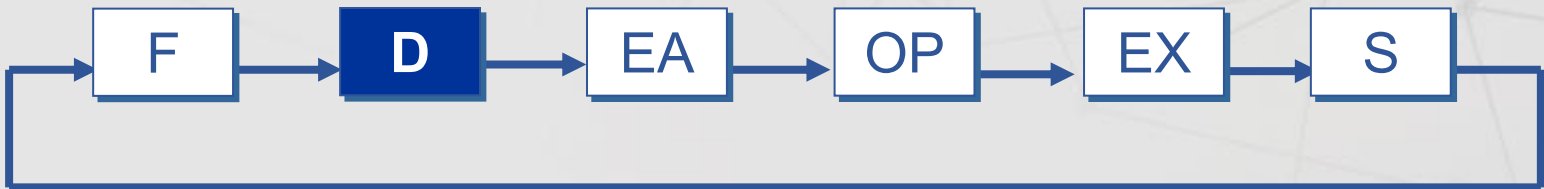
JMP (Register)



JMP (Register)



JMP (Register)



The logo of the University of Science and Technology of China (USTC) is a circular emblem. It features a blue outer ring with the university's name in Chinese characters at the top and in English at the bottom. The center of the logo contains a stylized white sailboat on a blue background, with the year '1958' positioned above it.

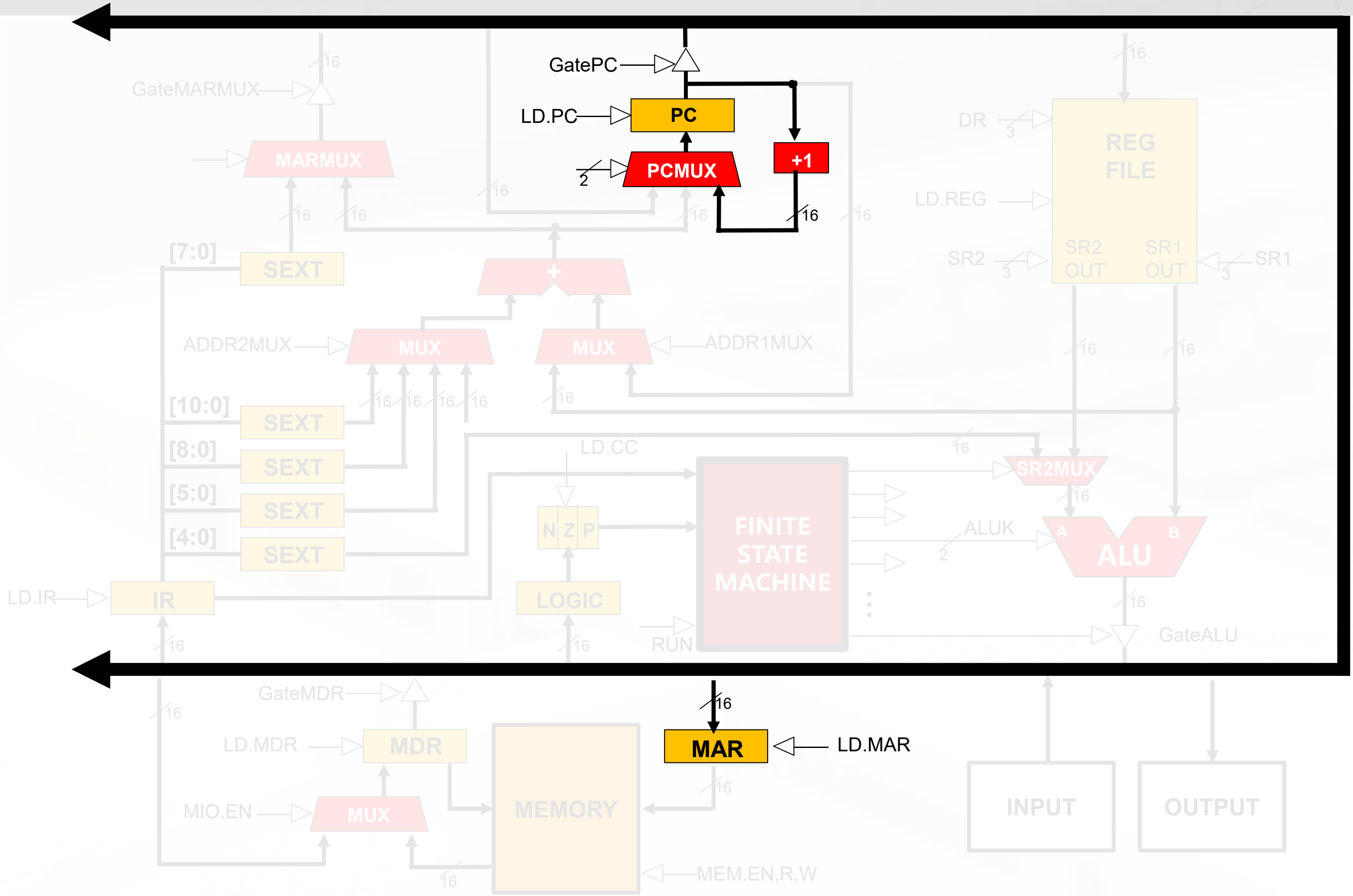
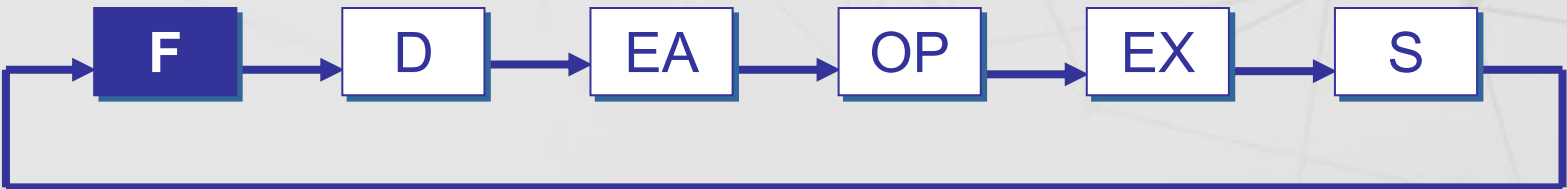


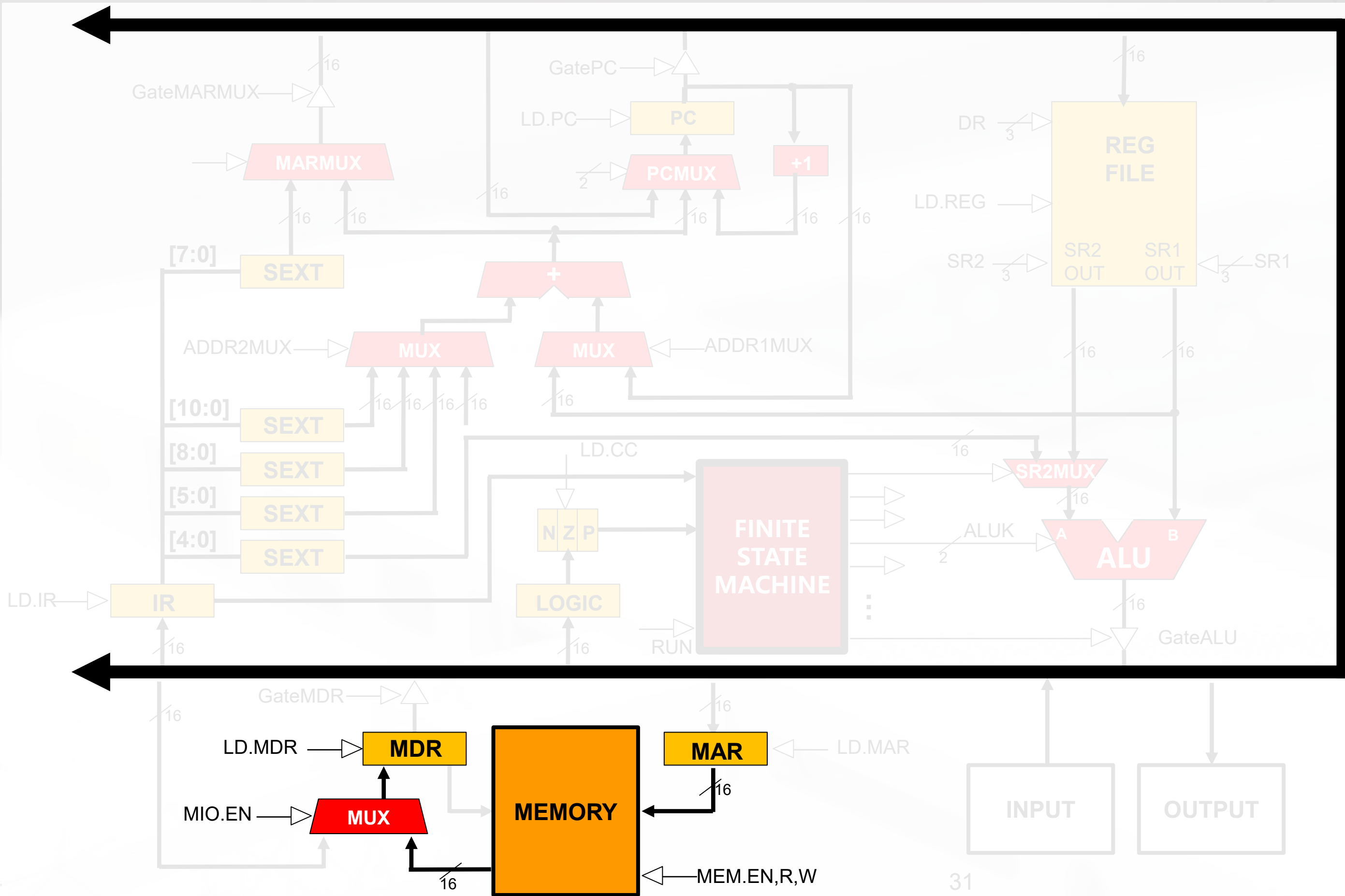
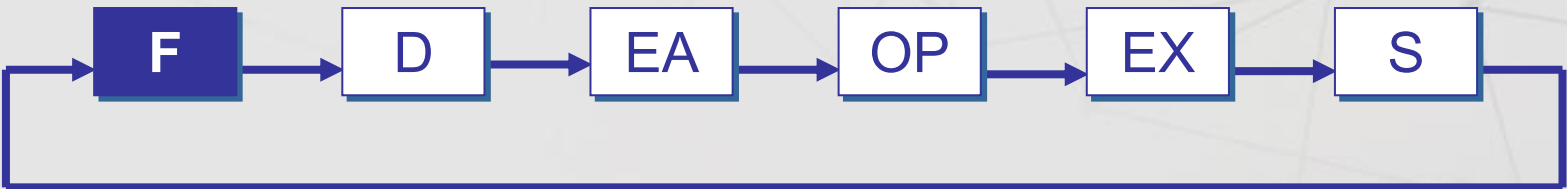


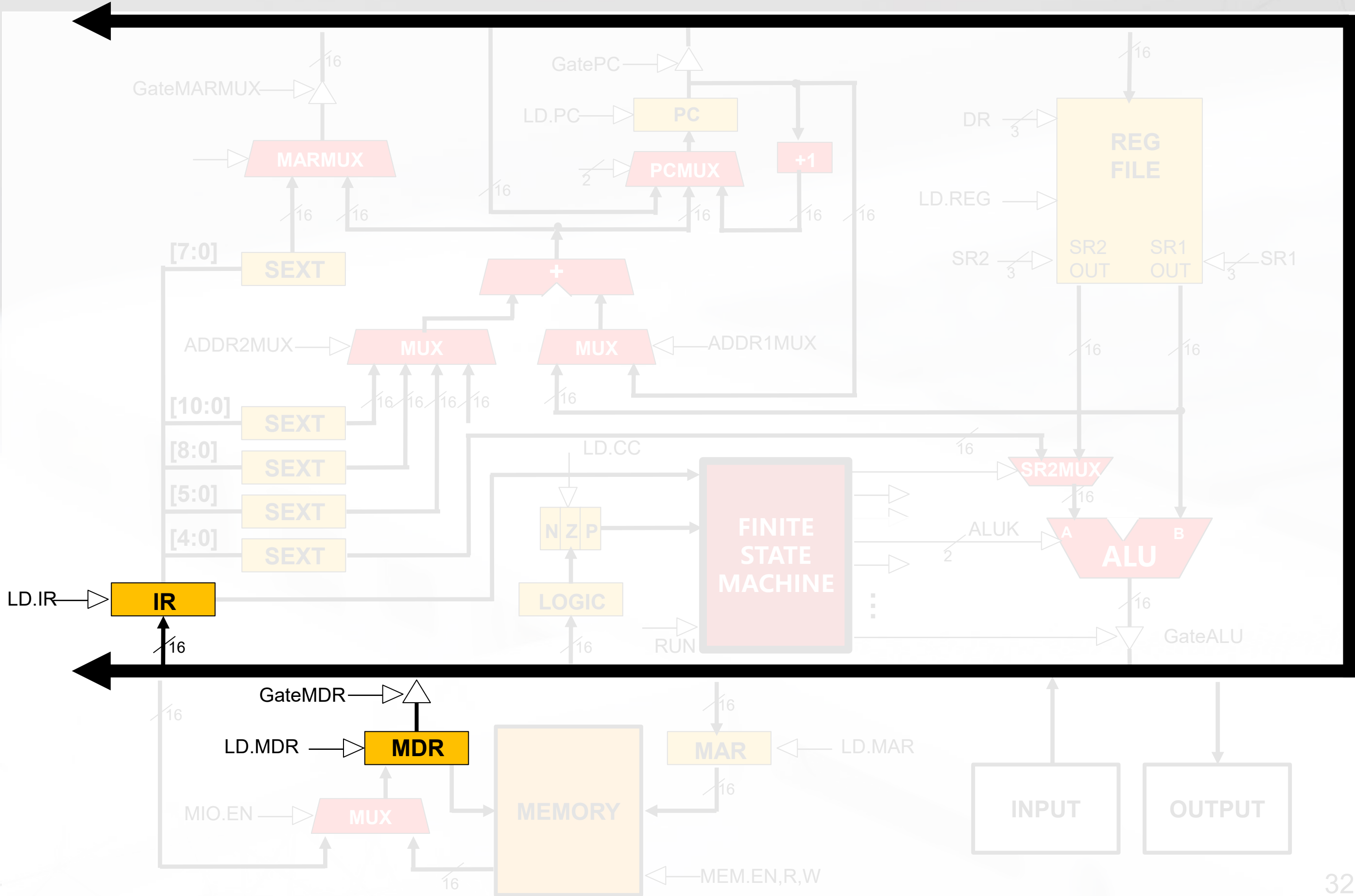
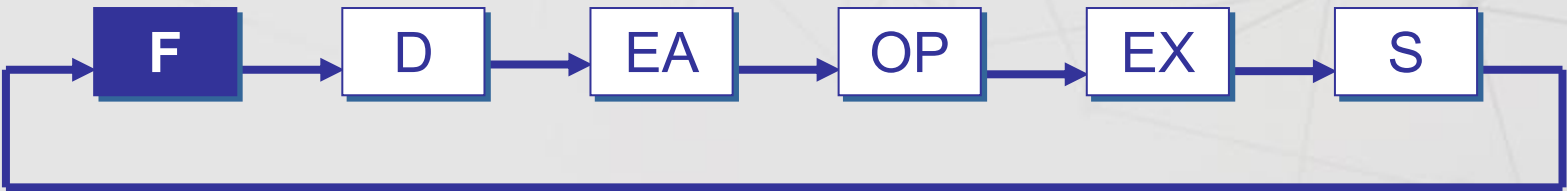
Calls a **service routine**, identified by 8-bit “trap vector.”

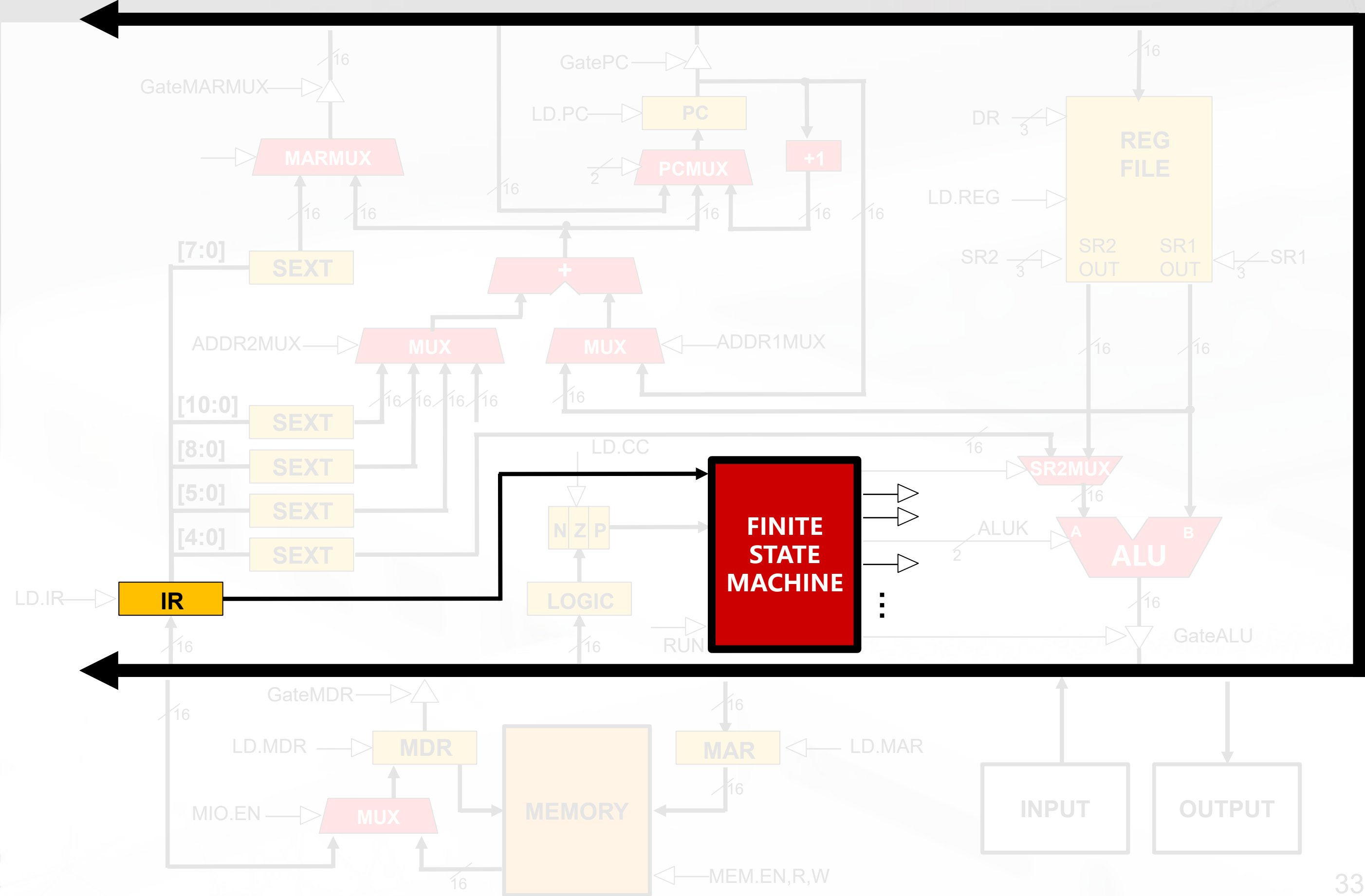
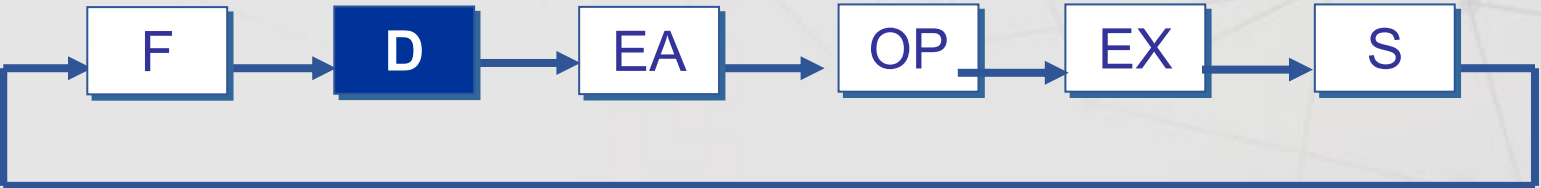
<i>vector</i>	<i>routine</i>
x23	input a character from the keyboard
x21	output a character to the monitor
x25	halt the program

When routine is done,
PC is set to the instruction following TRAP.
(We’ ll talk about how this works later.)

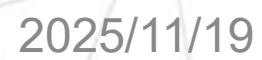




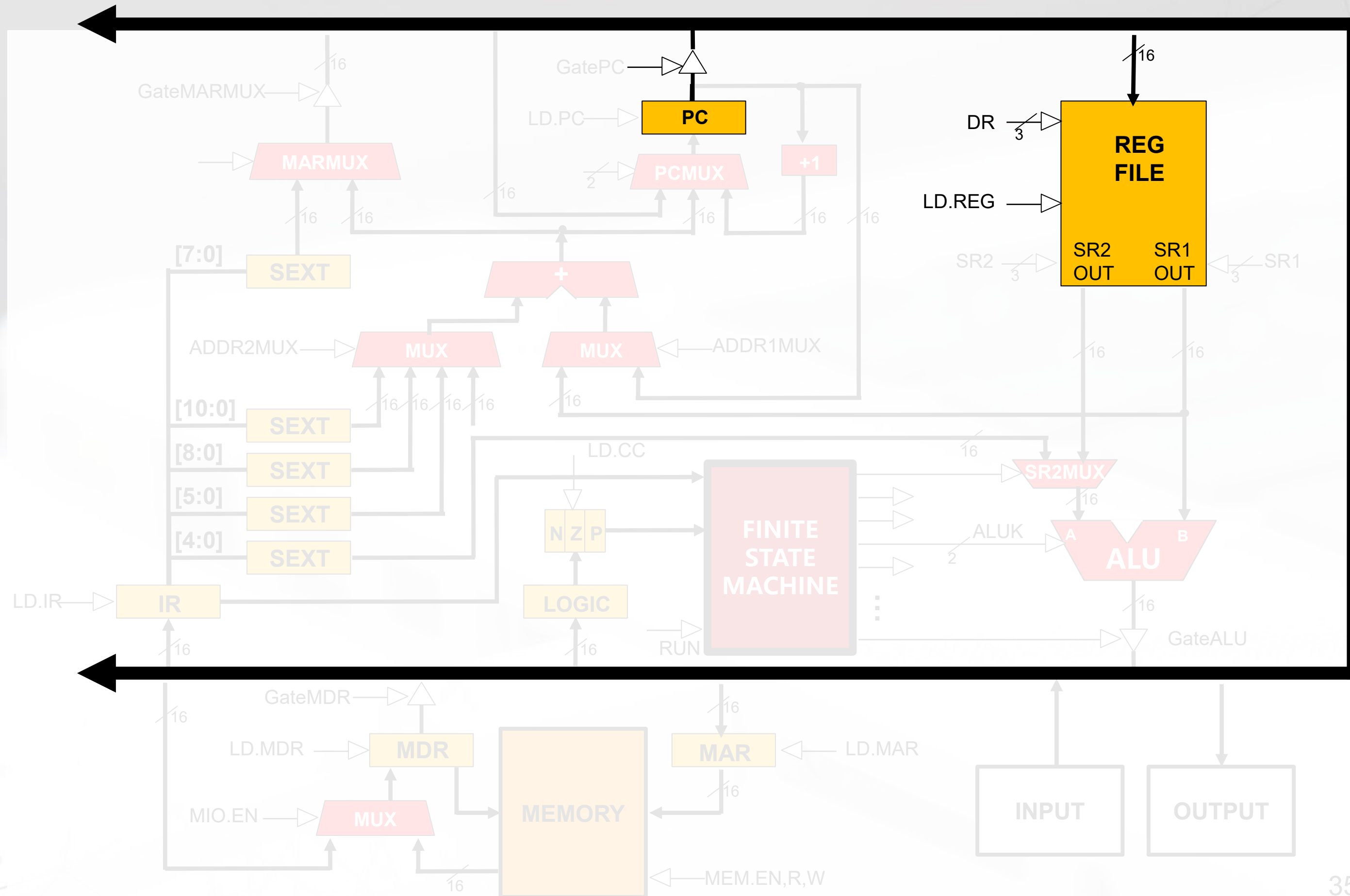


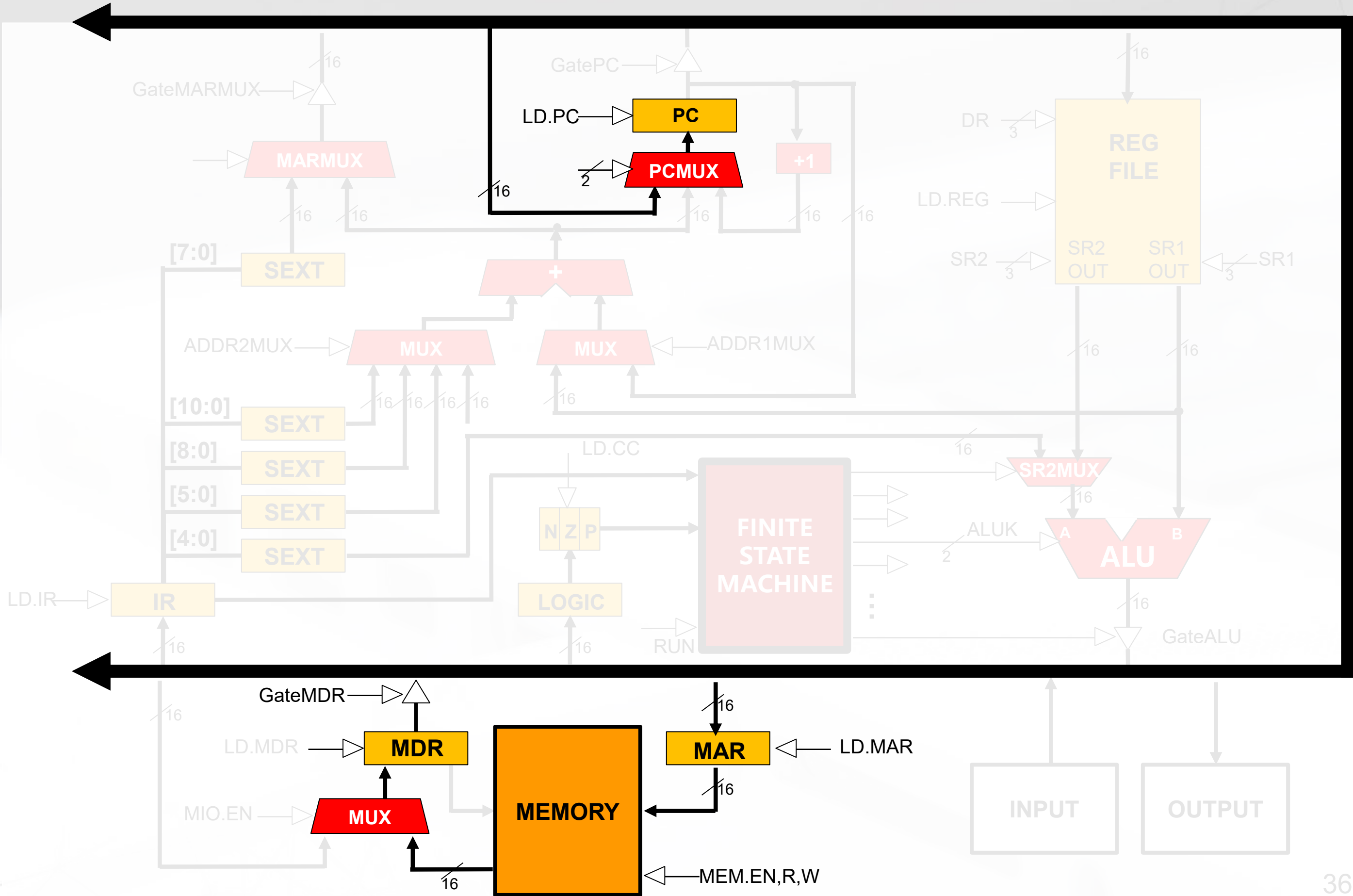
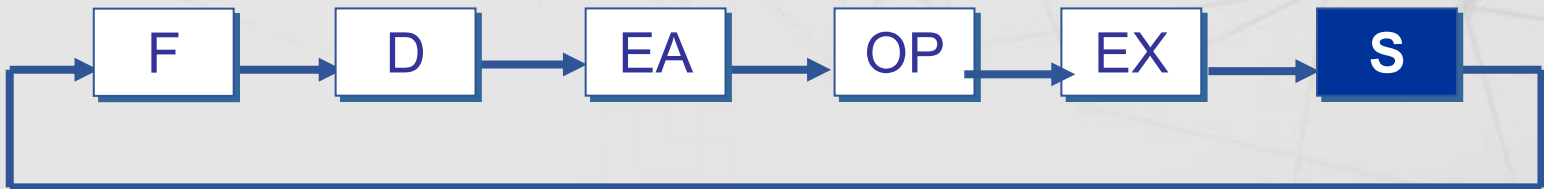


The logo of the University of Science and Technology of China (USTC) is a circular emblem. It features a blue outer ring with the university's name in Chinese characters at the top and in English at the bottom. The center of the logo is a stylized blue shield containing a white sailboat and the year '1958' above it.

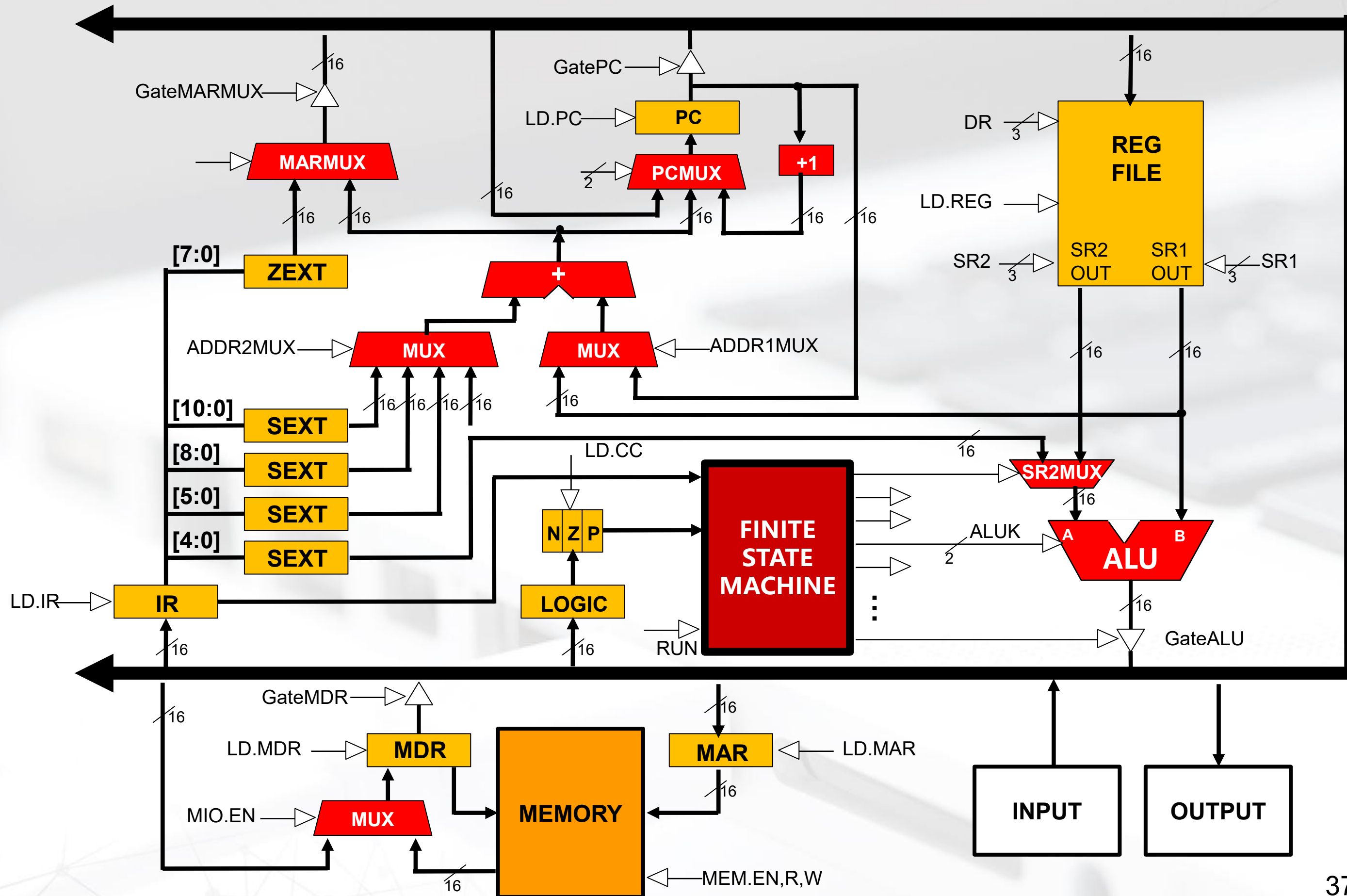


TRAP





LC-3 Data Path After Control Instruction



1 LC-3 ISA Overview

2 Operate Instructions and Data Path

3 Data Movement Instructions

4 Control Instructions

5 Another Example: Counting Occurrences of a Computer

6 The Data Path Revisited

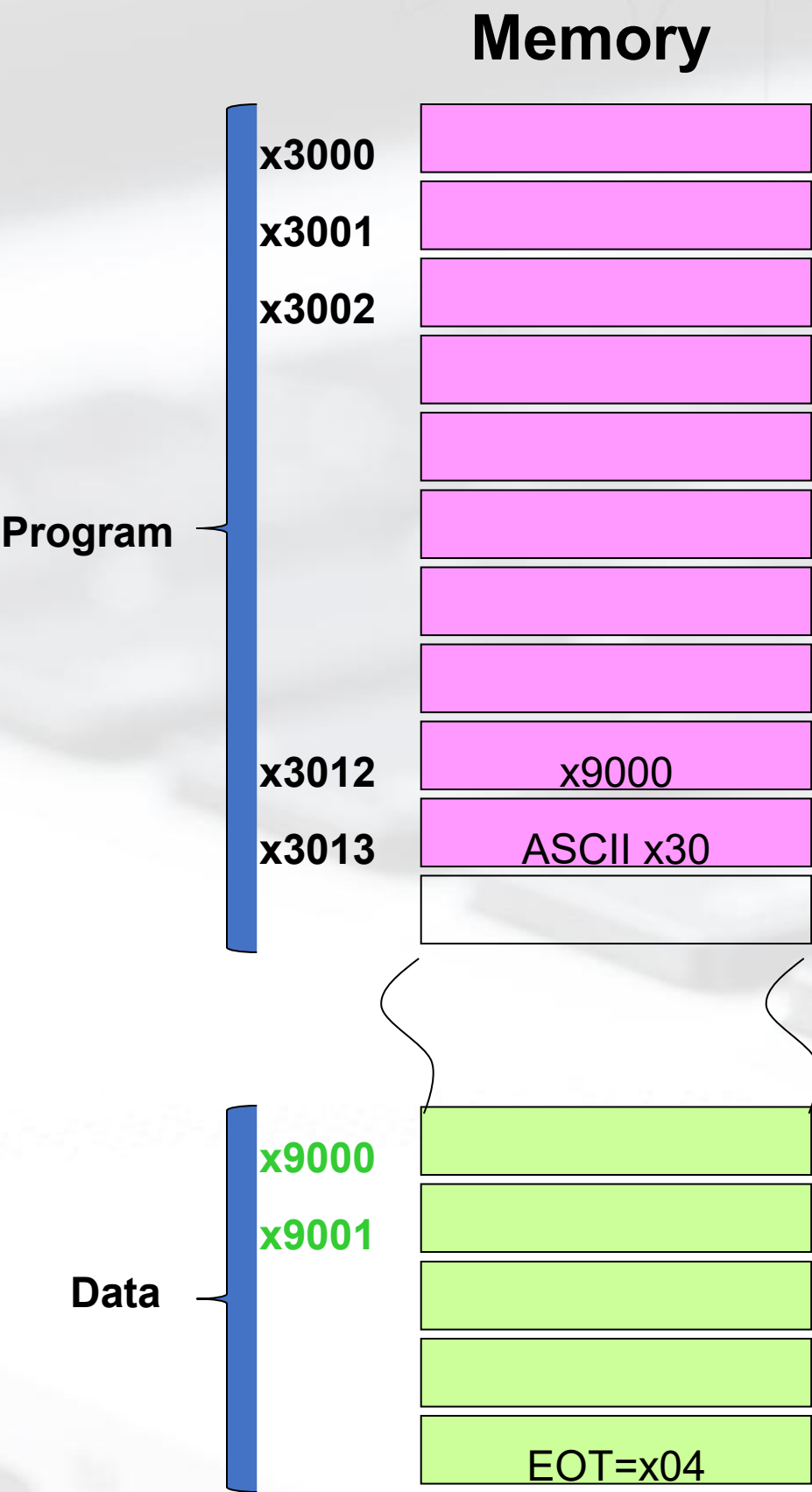
Counting the occurrences of a character in a file

- Program begins at location **x3000**
- Read character from keyboard
- Load each character from a “file”
 - File is a sequence of memory locations
 - Starting address of file is stored in the memory location immediately after the program
- If file character equals input character, increment counter
- End of file is indicated by a special ASCII value: **EOT (x04)**
- At the end, print the number of characters and halt
(assume there will be less than 10 occurrences of the character)

Counting the occurrences of a character in a file

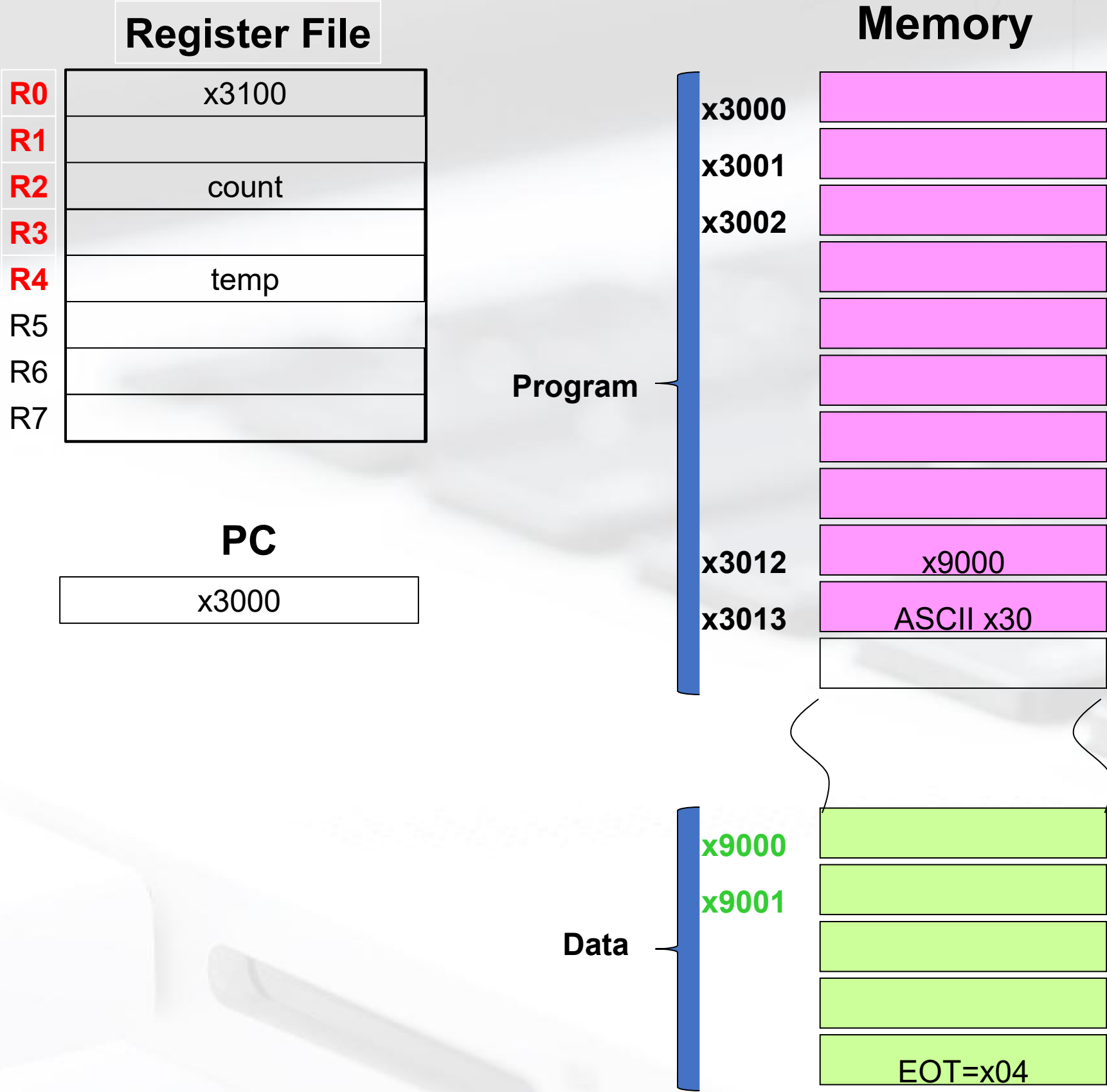
A special character used to indicate the end of a sequence is often called a **sentinel**.

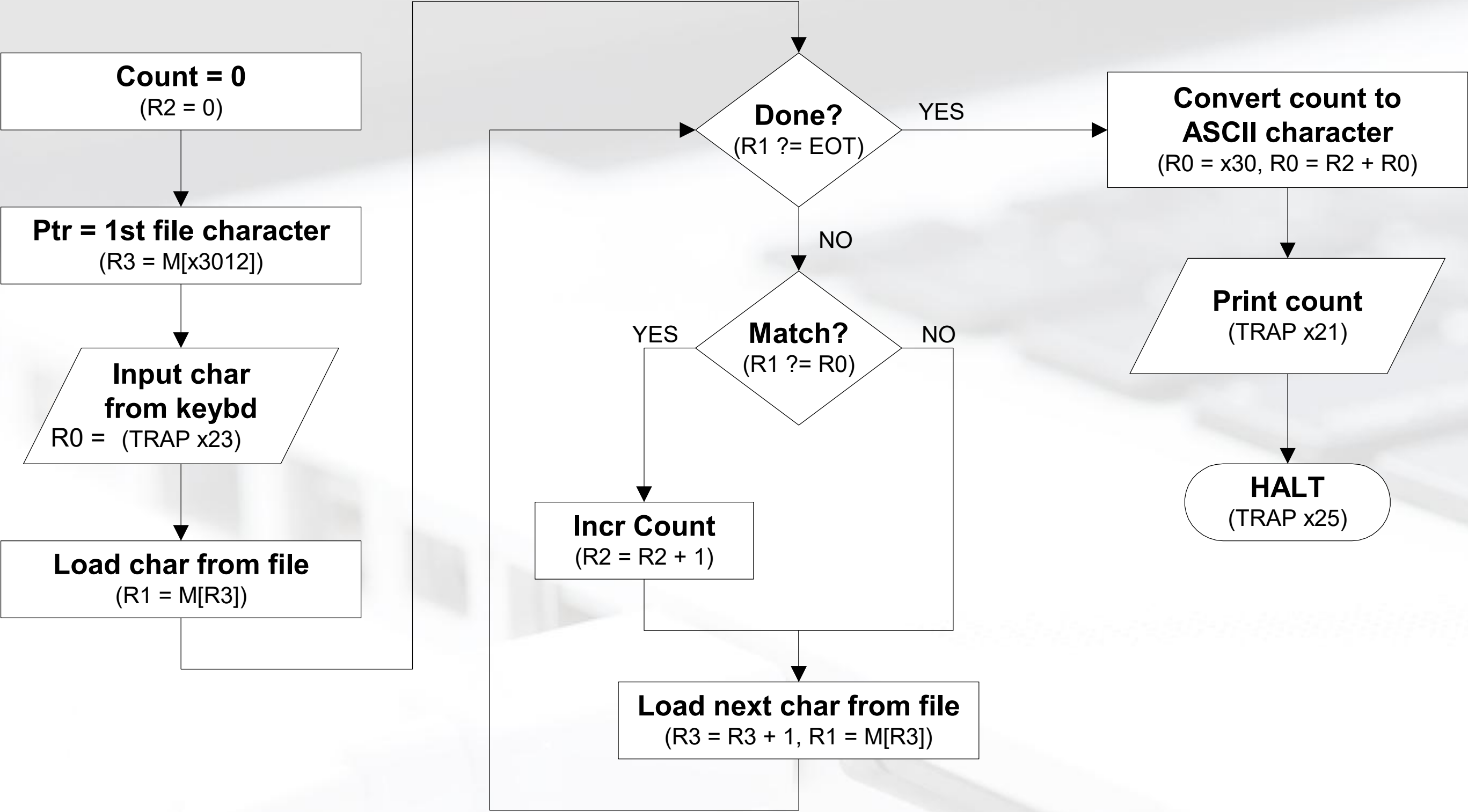
- Useful when you don't know ahead of time how many times to execute a loop.



Register

- R0: hold the character that is being counted (typed from keyboard)
- R1: hold, in turn, each character that we get from the file being examined
- R2: keep track of the number of occurrences
- R3: pointer(Ptr), at first, $M[x3012]=x9000$
- R4: temp, checking $R4= R1-ASCII(EOT)$





Program (1 of 2)

Address	Instruction													Comments	
x3000	0	1	0	1	0	1	0	0	1	0	1	0	0	0	$R2 \leftarrow 0$ (counter) AND R2,R2, #0
x3001	0	0	1	0	0	1	1	0	0	0	0	1	0	0	$R3 \leftarrow M[x3012]$ (ptr) LD R3, x3012 (LD R3, PTR)
x3002	1	1	1	1	0	0	0	0	0	0	1	0	0	0	Input to R0 (TRAP x23) TRAP x23 (GETC)
x3003	0	1	1	0	0	0	1	0	1	1	0	0	0	0	$R1 \leftarrow M[R3]$ LDR R1, R3, #0
x3004	0	0	0	1	1	0	0	0	0	1	1	1	1	0	$R4 \leftarrow R1 - 4$ (EOT) ADD R4,R1, #-4
x3005	0	0	0	0	0	1	0	0	0	0	0	0	1	0	If Z, goto x300E BRz x300E (BRz OUTPUT)
x3006	1	0	0	1	0	0	1	0	0	1	1	1	1	1	$R1 \leftarrow NOT R1$ NOT R1,R1
x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	$R1 \leftarrow R1 + 1$ ADD R1,R1,#1
x3008	0	0	0	1	0	0	1	0	0	1	0	0	0	0	$R1 \leftarrow R1 + R0$ ADD R1,R1,R0
x3009	0	0	0	0	1	0	1	0	0	0	0	0	0	0	If N or P, goto x300B BRnp x300B (BRnp GETCHAR)

Program (2 of 2)

Address	Instruction															Comments	
x300A	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1	<i>R2 ← R2 + 1</i> <i>ADD R2,R2,#1</i>
x300B	0	0	0	1	0	1	1	0	1	1	1	0	0	0	0	1	<i>R3 ← R3 + 1</i> <i>ADD R3,R3,#1</i>
x300C	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	<i>R1 ← M[R3]</i> <i>LDR R1,R3,#0</i>
x300D	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1	0	<i>Goto x3004</i> <i>BRnzp x3004 (BRnzp TEST)</i>
x300E	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	<i>R0 ← M[x3013]</i> <i>LD R0,x3013 (LD R0, ASCII)</i>
x300F	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	<i>R0 ← R0 + R2</i> <i>ADD R0,R0,R2</i>
x3010	1	1	1	1	0	0	0	0	0	0	1	0	0	0	0	1	<i>Print R0</i> <i>TRAP x21 (OUT)</i>
x3011	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	<i>HALT</i> <i>TRAP x25 (HALT)</i>
X3012	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	Starting Address of File (X9000)
x3013	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	<i>ASCII x30 ('0')</i>

- 1 LC-3 ISA Overview**
- 2 Operate Instructions and Data Path**
- 3 Data Movement Instructions**
- 4 Control Instructions**
- 5 Another Example: Counting Occurrences of a Computer**
- 6 The Data Path Revisited**

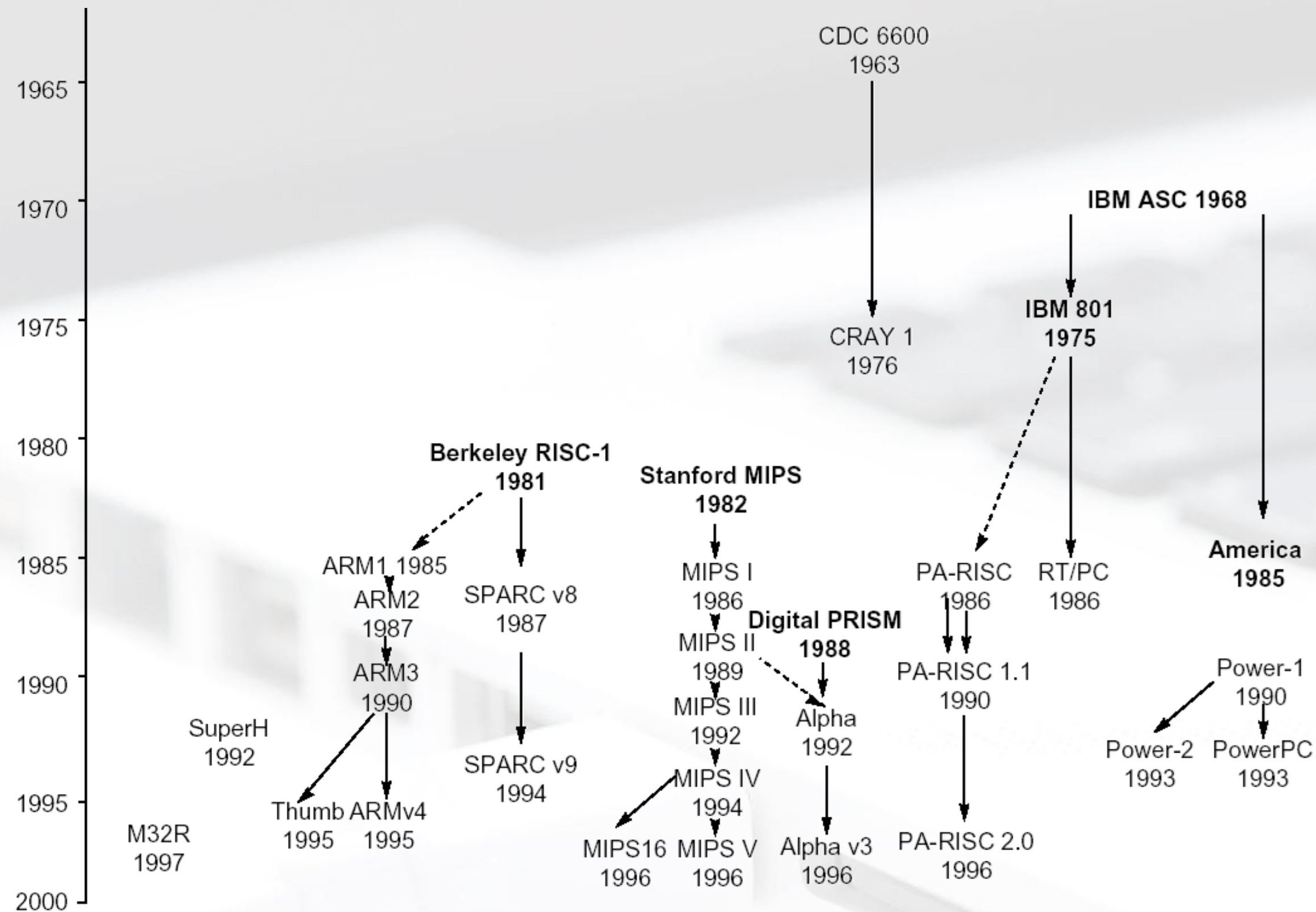
Instruction Set Architecture (ISA)

- Computer's native operations called **instructions**.
- Job of a CPU (Central Processing Unit, aka Core): execute instructions
 - Instructions: CPU's primitives operations
 - Instructions performed one after another in sequence
 - Each instruction does a small amount of work (a tiny part of a larger program) .
 - Each instruction has an operation applied to operands, and might be used change the sequence of instruction.
- Instruction set architecture (ISA) specifies the set of commands (instructions) a computer can execute
- Hardware registers provide a few very fast variables for instructions to operate on

Instruction Set Architecture (ISA)

- The instruction set defines all the valid instructions.
- CPUs belong to “families,” each implementing its own set of instructions
- CPU’ s particular set of instructions implements an Instruction Set Architecture (ISA)
- Examples:
 - ARM,
 - Intel x86
 - MIPS
 - RISC-V
 - IBM/Motorola PowerPC (old Mac)
 - Intel IA64,
 - . . .

Instruction set architecture evolution



运算指令

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD	0	0	0	1	DR		SR1		0	0	0	SR2				
ADD	0	0	0	1	DR		SR1		1	Imm5						
AND	0	1	0	1	DR		SR1		0	0	0	SR2				
AND	0	1	0	1	DR		SR1		1	Imm5						
NOT	1	0	0	1	DR		SR1		1	1	1	1	1	1	1	
Reserved	1	1	0	1												

控制指令

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR	0	0	0	0	n	z	p	PCOffset9								
JSR	0	1	0	0	1	PCOffset11										
JSRR	0	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	0
RTI	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
JMP	1	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	0
RET	1	1	0	0	0	0	0	1	1	1	0	0	0	0	0	0
TRAP	1	1	1	1	0	0	0	0	TrapVector8							

移动数据指令

取数指令

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LD	0	0	1	0	DR			PCoffset9								
LDR	0	1	1	0	DR			BaseR		PCoffset6						
LDI	1	0	1	0	DR			PCoffset9								
LEA	1	1	1	0	DR			PCoffset9								

存数指令

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ST	0	0	1	1	SR			PCOffset9								
STR	0	1	1	1	SR			BaseR		PCOffset6						
STI	1	0	1	1	SR			PCOffset9								

Instruction Set Architecture (ISA) vs. Finite State Automata

[illegible]

Figure A.2 Format of the entire LC-3 instruction set. Note: + indicates instructions that modify condition codes

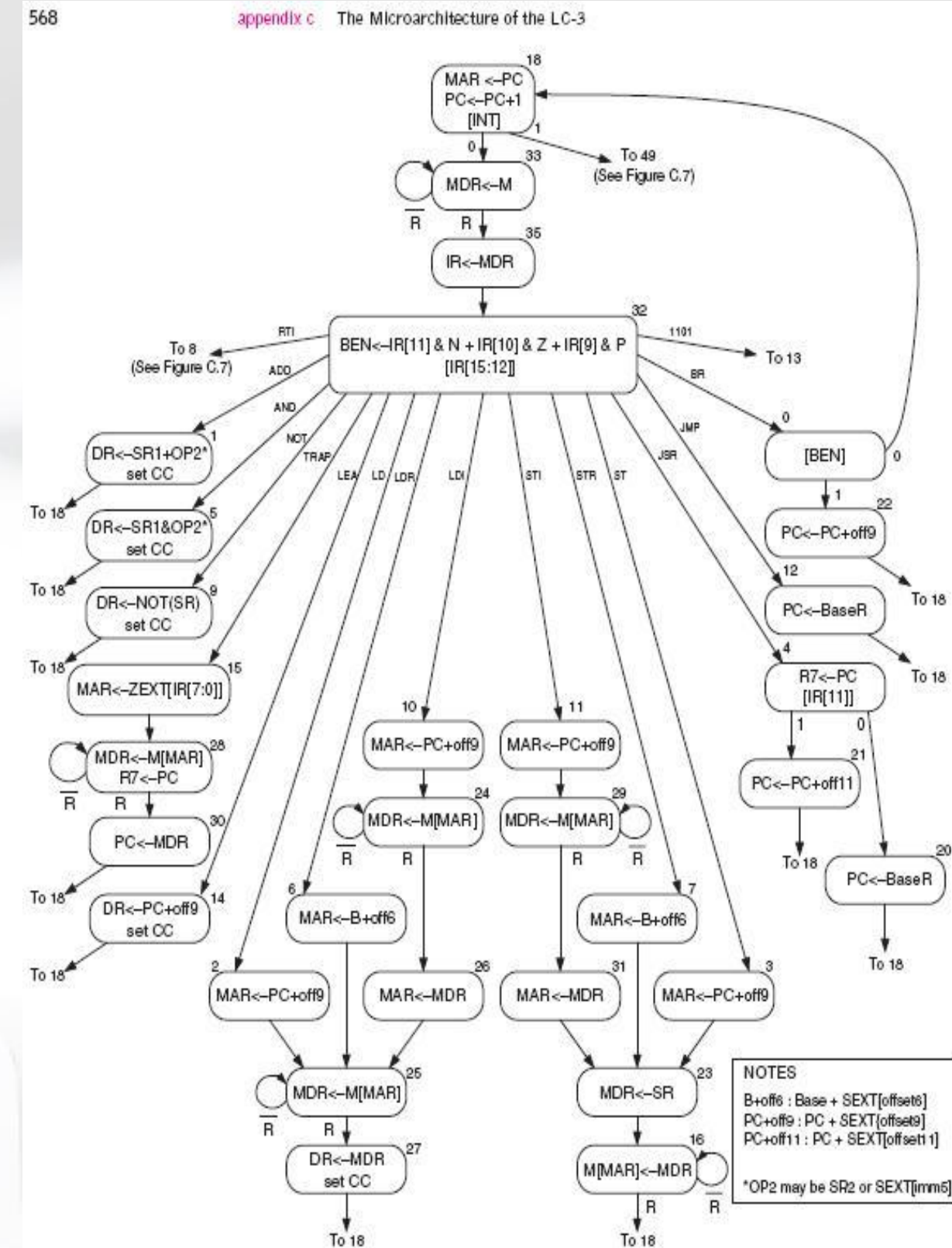
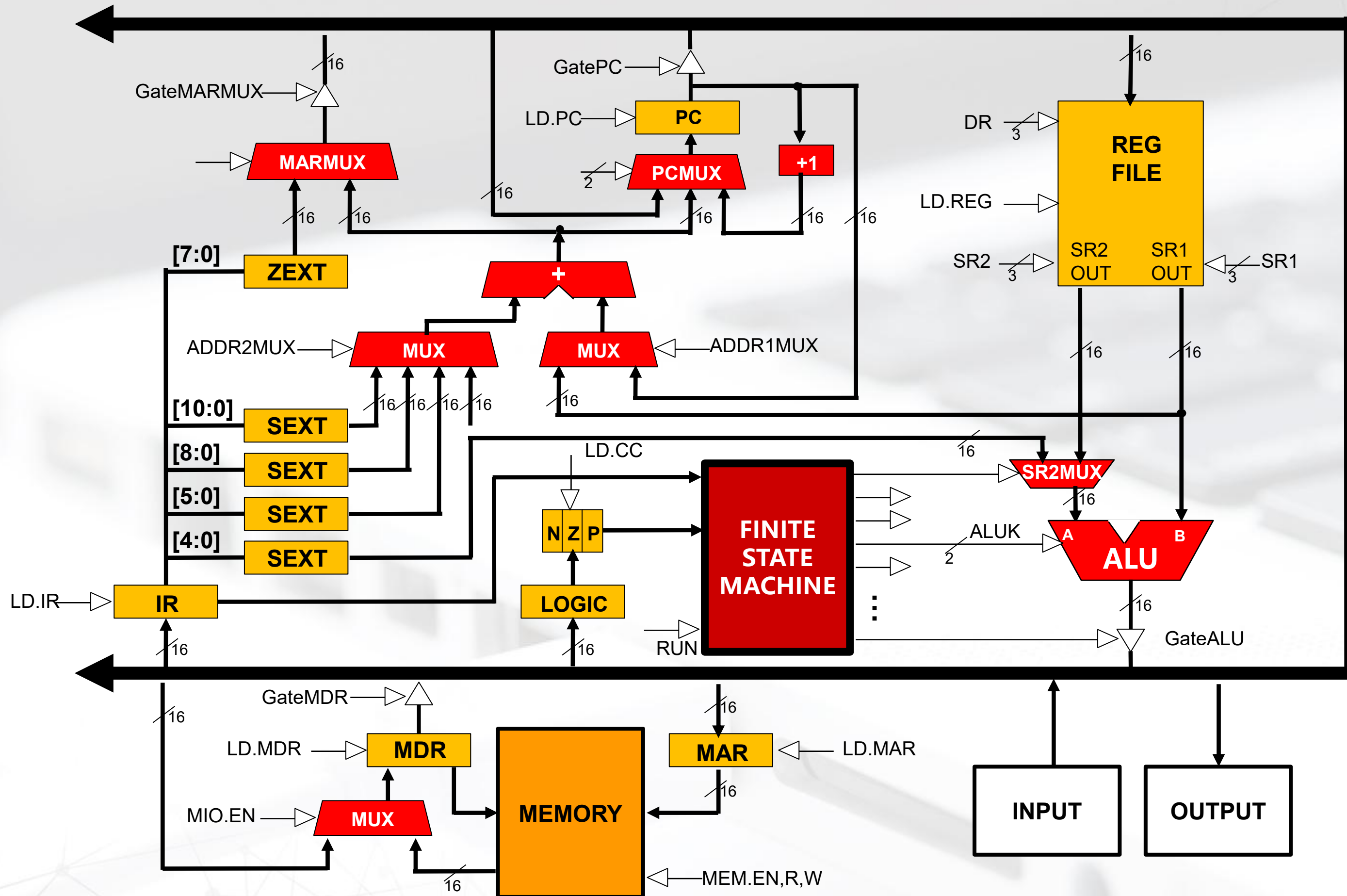


Figure C.2 A state machine for the LC-3

LC-3 Data Path



Basic Components of the Data Path

The Global bus

- special set of wires that carry a 16-bit signal to many components
- inputs to the bus are “**tri-state devices**,” that only place a signal on the bus when they are enabled
- only one (16-bit) signal should be enabled at any time
 - control unit decides which signal “drives” the bus
- any number of components can read the bus
 - register only captures bus data if it is write-enabled by the control unit

Memory

- Control and data registers for memory and I/O devices
- memory: MAR, MDR (also control signal for read/write)

Basic Components of the Data Path

The ALU

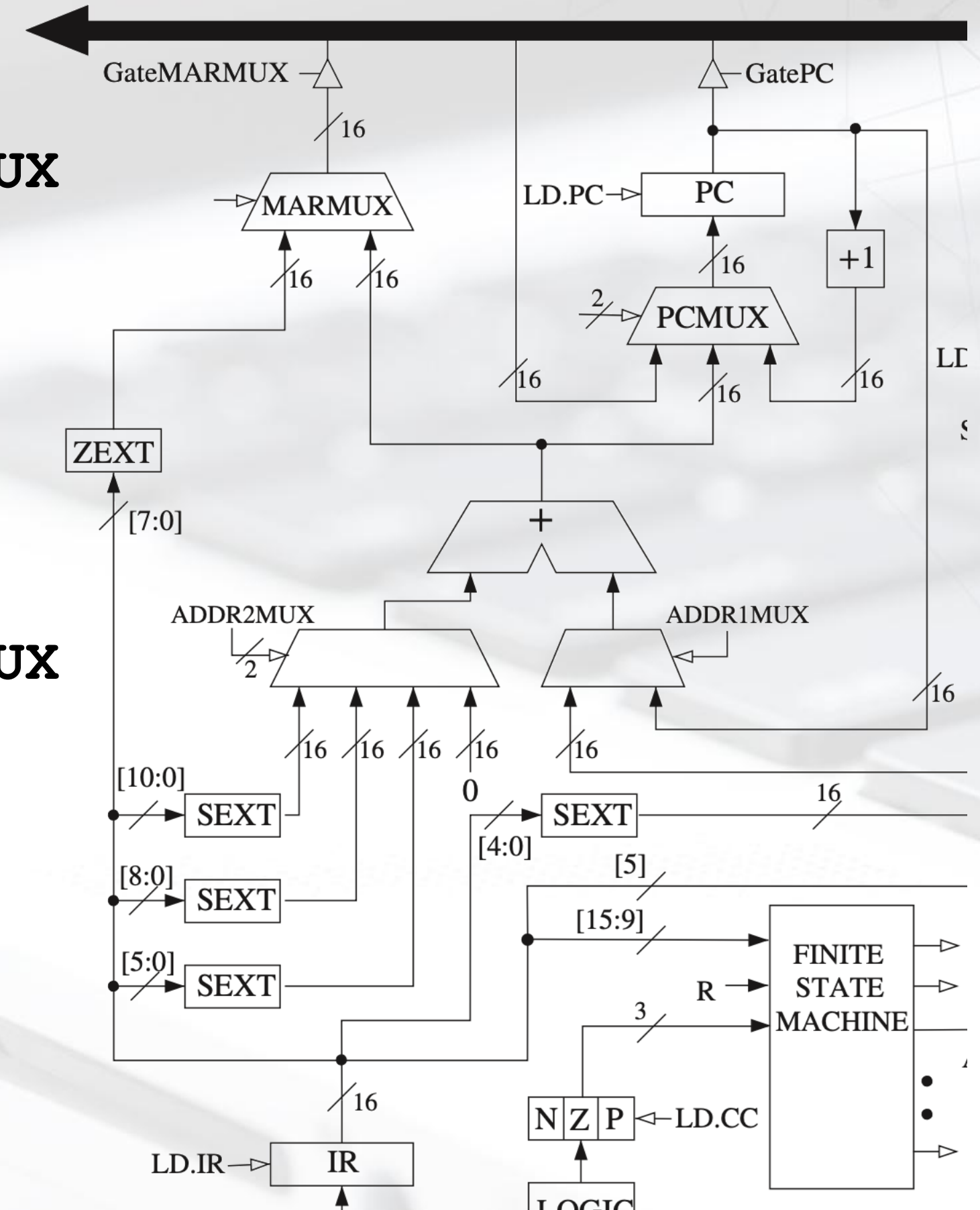
- Accepts inputs from register file and from sign-extended bits from IR (immediate field).
- Output goes to bus.
 - used by condition code logic, register file, memory

The Register File

- Two read addresses (SR1, SR2), one write address (DR)
- Input from bus
 - result of ALU operation or memory read
- Two 16-bit outputs
 - used by ALU, PC, memory address
 - data for store instructions passes through ALU

1. PC+1 - FETCH stage
2. Address adder - BR, JMP
3. bus - TRAP (discussed later)

1. Address adder – LD/ST, LDR/STR
2. Zero-extended IR[7:0] -- TRAP (discussed later)



Basic Components of the Data Path

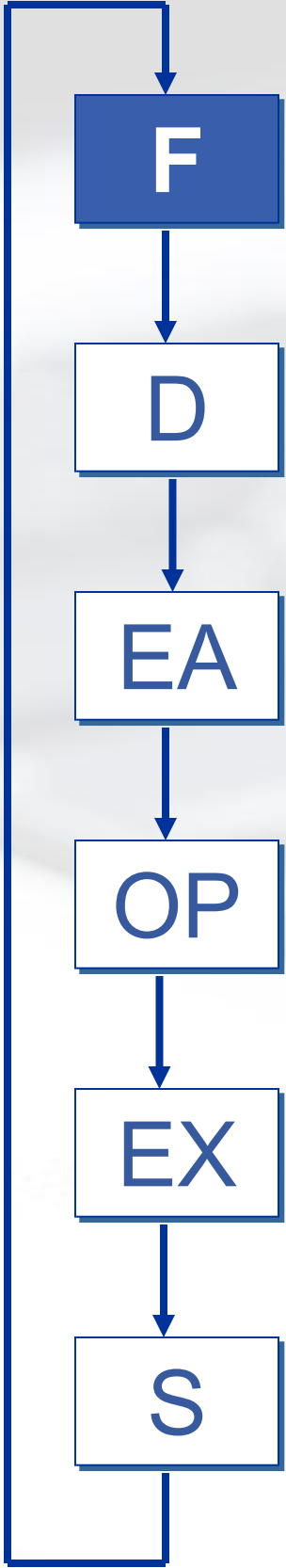
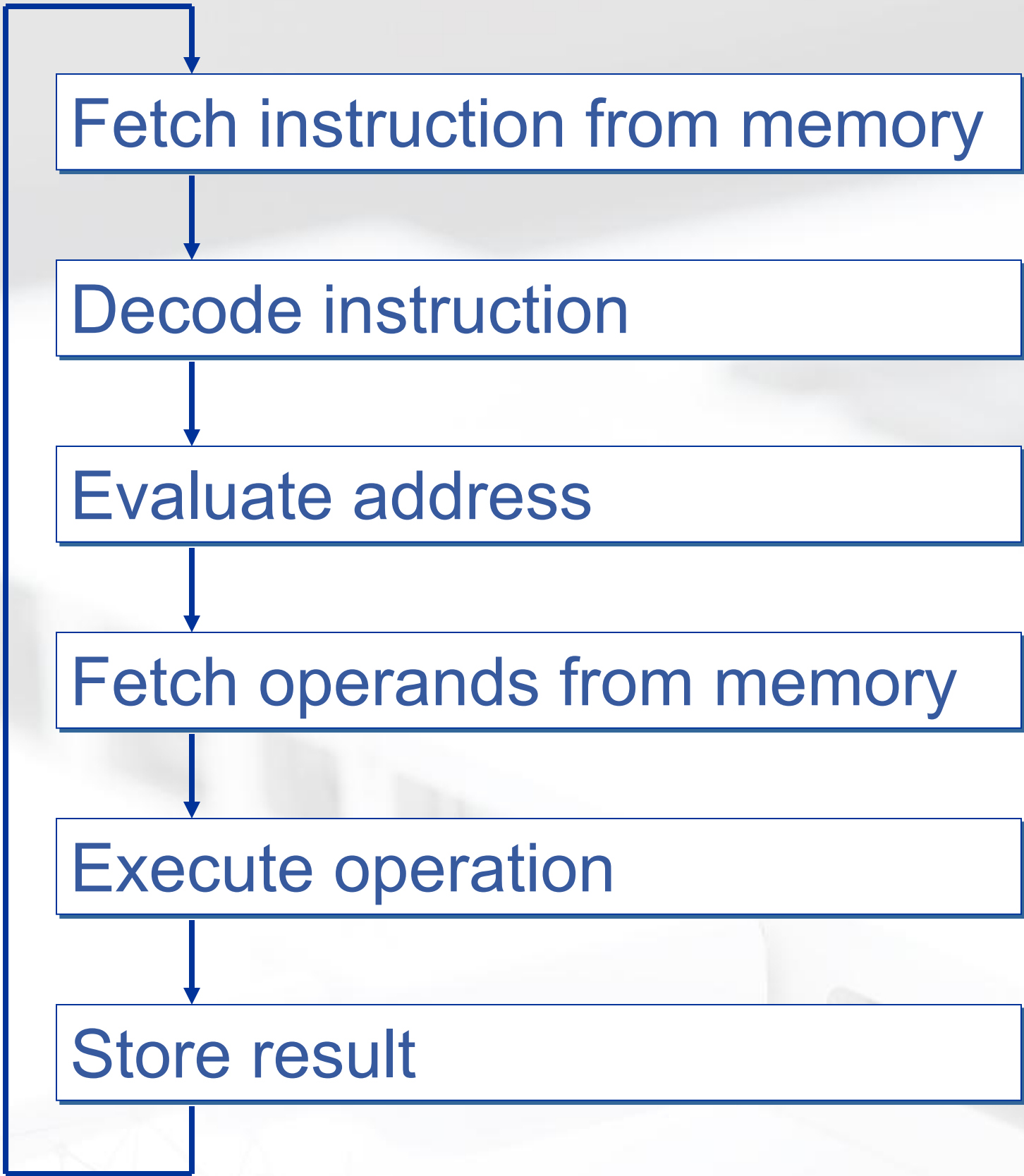
Condition Code Logic

- Looks at value on bus and generates N, Z, P signals
- Registers set only when control unit enables them (LD.CC)
 - only certain instructions set the codes (ADD, AND, NOT, LD, LDI, LDR, LEA)

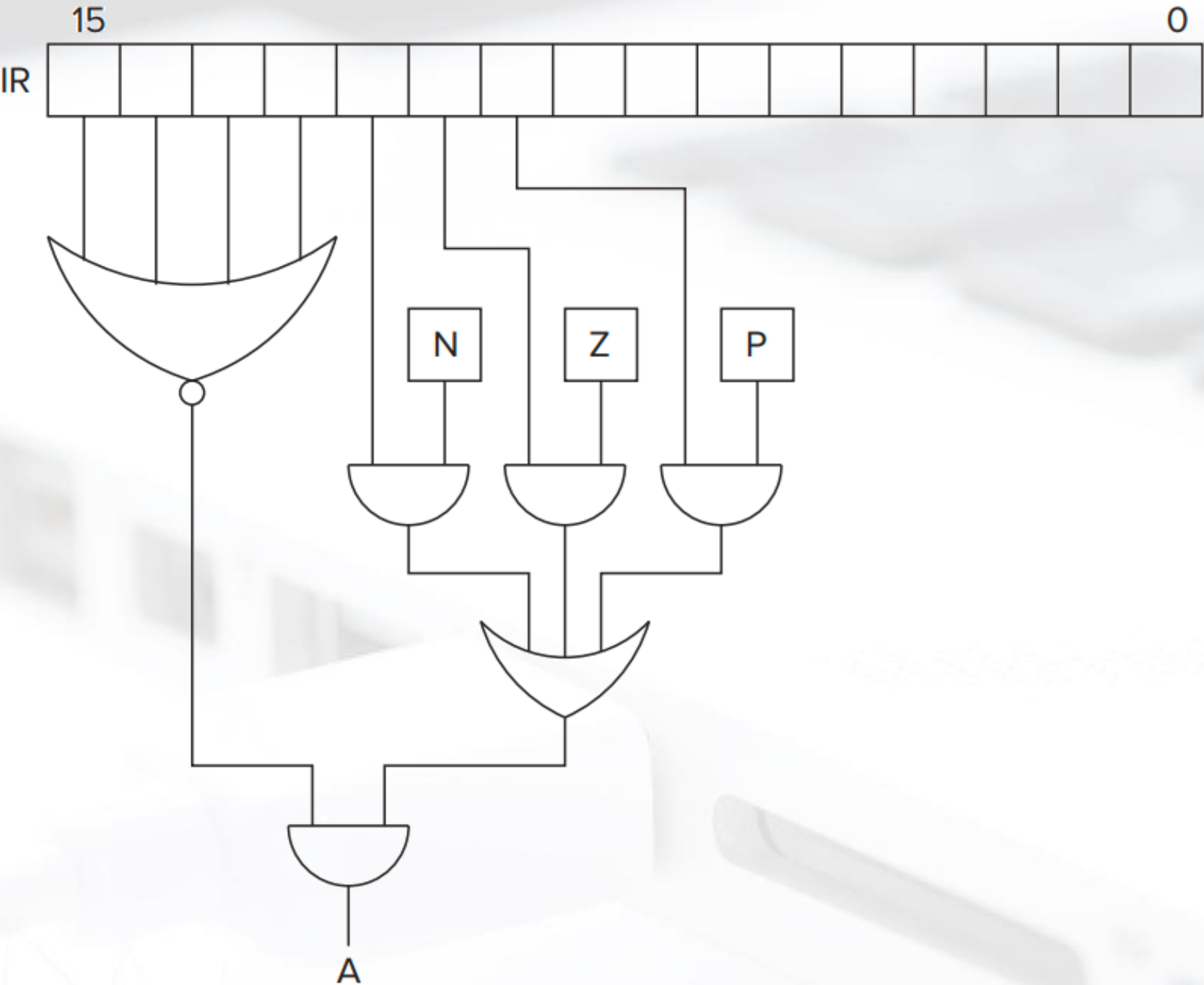
Control Unit – Finite State Machine

- On each machine cycle, changes control signals for next phase of instruction processing
 - who drives the bus? (GatePC, GateALU, ...)
 - which registers are write enabled? (LD.IR, LD.REG, ...)
 - which operation should ALU perform? (ALUK)
 - ...
- Logic includes decoder for opcode, etc.

The Instruction Cycle Specific to the LC-3



5.40 The following logic diagram shows part of the control structure of the LC-3 machine. What is the purpose of the signal labeled A?



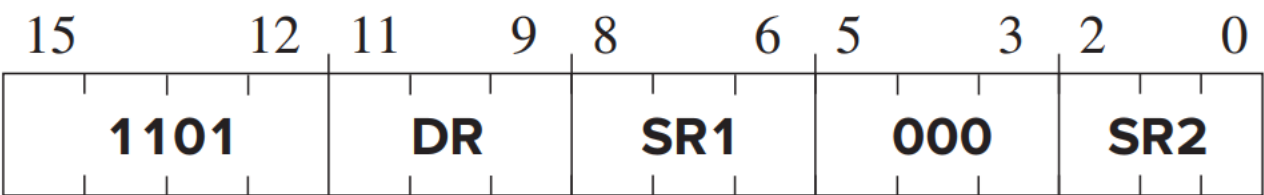
- What information does Y provide?
- The signal X is the control signal that gates the gated D latch. Is there an error in the logic that produces X?



Exercise



★5.51 An aggressive young engineer decides to build and sell the LC-3 but is told that if he wants to succeed, he really needs a SUBTRACT instruction. Given the unused opcode 1101, he decides to specify the SUBTRACT instruction as follows:



The instruction is defined as: $DR \leftarrow SR2 - SR1$, and the condition codes are set. Assume DR, SR1, and SR2 are all different registers. To accomplish this, the engineer needs to add three states to the state machine and a mux and register A to the data path. The modified state machine is shown below, and the modified data path is shown on the next page. The mux is controlled by a new control signal SR2SEL, which selects one of its two sources.

SR2SEL/1: SR2OUT, REGISTER_A

Your job:

For the state machine shown below, fill in the empty boxes with the control signals that are needed in order to implement the SUBTRACT instruction.

For the data path, fill in the value in register A.

